# Chapter 19 — Cheat Sheet and Glossary

This chapter is the book on one table. Chapters 1–18 teach each topic with runnable projects; this chapter **looks things up**. You come here when you remember that something exists but not its exact name, default or package: "which starter brings `@WebMvcTest` in Boot 4?", "what does `REQUIRES_NEW` do when the caller rolls back?", "which property turns on virtual threads?", "is it `hasRole("ADMIN")` or `hasRole("ROLE_ADMIN")`?".

Every row links to the section that explains it, so a sheet is never the end of the road: if a line is not enough, follow the **§** link. Nothing here is new material; where a sheet quotes a default or a behaviour, a test in `code/cheatsheet-checks/` proves it (§19.15).

**What you'll find**

1. Versions of the Boot 4.1.1 stack at a glance (§19.2).
2. **Annotations**: every common one → module / starter → purpose → section (§19.3).
3. **Starters** in Boot 4.1, including renamed, deprecated and removed ones (§19.4).
4. **Properties** with their defaults: web, data, cache, security, actuator, observability, Kafka, threads, HTTP clients, API versioning (§19.5).
5. `@Transactional`: attributes, **propagation**, isolation and **rollback** tables (§19.6).
6. **Bean scopes** and the **lifecycle order** (§19.7).
7. **Spring Security 7 DSL** snippets that compile: form login, JWT resource server, method security, CORS/CSRF (§19.8).
8. **Testing** annotations and slices (§19.9), **Reactor** operators (§19.10), **Actuator** endpoints (§19.11).
9. **Which tool for which job** decision tables (§19.12) and a **Boot 3 → 4 checklist** (§19.13).
10. An **interview index** into Chapter 18 (§19.14), the **proofs** (§19.15) and an **A–Z glossary** of more than 200 terms (§19.16).

Versions: Spring Boot **4.1.1** → Spring Framework **7.0.9**, Spring Security **7.1.1**, Java **25** (17 minimum). "(Boot 4)", "(F7)" and "(S7)" mark things that are new or changed in Boot 4, Framework 7 or Security 7.

---

## 19.1 How to use this chapter

**How the sheets are organised**

| You want to... | Go to |
|---|---|
| find the name, jar or purpose of an annotation | §19.3 |
| add a dependency | §19.4 |
| set or look up a property and its default | §19.5 |
| pick a propagation / isolation / rollback rule | §19.6 |
| remember the bean lifecycle order | §19.7 |
| copy a security configuration | §19.8 |
| choose a test annotation | §19.9 |
| find a Reactor operator | §19.10 |
| know what an Actuator endpoint shows | §19.11 |
| choose between two technologies | §19.12 |
| upgrade a Boot 3 app | §19.13 |
| revise for an interview | §19.14 |
| look up a word | §19.16 |

**Conventions**

- **Module / starter**: the jar that contains the class, and the Boot starter that brings it. "core" means `spring-context` / `spring-beans`, which every starter includes.
- **§N.M** links go to the teaching section of Chapters 1–18. Follow them for the explanation, the trade-offs and the proof.
- Defaults quoted in these sheets come from the Boot 4.1.1 configuration metadata (the `spring-configuration-metadata.json` files inside the Boot jars) or from the chapter that measured them. The ones marked **✔ proved** are asserted by a test in §19.15.
- Properties are written in `.properties` form. The same keys work in YAML and as environment variables (§3.4).

---

## 19.2 Versions at a glance

What `spring-boot-starter-parent` **4.1.1** puts on your classpath (from the `spring-boot-dependencies-4.1.1` BOM). You choose one number, the Boot version, and Boot chooses the rest (§1.4).

| Component | Version | Notes |
|---|---|---|
| Java | 17 minimum, 25 recommended | this book: Java 25 |
| Spring Framework | 7.0.9 | Jakarta EE 11 (Servlet 6.1, JPA 3.2, Bean Validation 3.1) |
| Spring Security | 7.1.1 | Authorization Server now inside Spring Security (§11.14) |
| Spring Data | 2026.0.1 release train | JPA, JDBC, MongoDB, Redis, R2DBC, REST |
| Hibernate ORM / Validator | 7.4.5 / 9.1.3 | |
| Jackson | 3.1.5 (`tools.jackson.*`) | annotations still `com.fasterxml.jackson.annotation` (§5.5) |
| Tomcat / Jetty | 11.0.24 / 12.1.12 | Undertow removed |
| HikariCP / H2 / PostgreSQL JDBC | 7.0.2 / 2.4.240 / 42.7.13 | |
| Reactor | 2025.0.7 train (Reactor Core 3.8) | |
| Micrometer / Micrometer Tracing / OpenTelemetry | 1.17.1 / 1.7.1 / 1.62.0 | |
| Spring for Apache Kafka / kafka-clients | 4.1.1 / 4.2.1 | |
| Spring AMQP | 4.1.1 | |
| Flyway / Liquibase | 12.4.0 / 5.0.3 | Liquibase 5 Community is FSL-licensed (§9.4) |
| JUnit / Mockito / AssertJ / Awaitility | 6.0.3 / 5.23.0 / 3.27.7 / 4.3.0 | |
| Testcontainers | 2.0.5 | new artifact names `testcontainers-<module>` (§12.14) |
| Caffeine / Lettuce | 3.2.4 / 7.5.2 | |
| **Not managed by Boot** | | import their own BOM |
| Spring Cloud | 2025.1.x "Oakwood" (2025.1.2+ for Boot 4.1) | §16.2 |
| Spring AI | 2.0.x (2.0.1) | §17.1 |
| springdoc-openapi | 3.x (3.1.1) | §5.12 |
| Resilience4j | `resilience4j-spring-boot4` 2.4.0 | §16.8 |

---

## 19.3 Annotations cheat sheet

The annotations of this book, grouped by area. "Needs" names the switch without which the annotation silently does nothing — the most common reason an annotation "doesn't work".

### Core container and beans

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@Component` | core | generic bean found by component scanning | §2.3 |
| `@Service` | core | `@Component` for business logic (no extra behaviour) | §2.3 |
| `@Repository` | core | `@Component` for data access + exception translation to `DataAccessException` | §2.3, §7.5 |
| `@Controller` / `@RestController` | spring-web / `-webmvc`, `-webflux` | web handler; `@RestController` = `@Controller` + `@ResponseBody` | §5.3 |
| `@Configuration` | core | class of `@Bean` methods; full mode = CGLIB subclass (`proxyBeanMethods = true`) | §2.3 |
| `@Bean` | core | factory method registering its return value as a bean; name = method name | §2.3 |
| `@ComponentScan` | core | scan a package tree for components (inside `@SpringBootApplication`) | §1.8 |
| `@Import` | core | pull in another configuration class, selector, registrar or `BeanRegistrar` | §2.3 |
| `@Autowired` | core | inject here; optional on a single constructor | §2.4 |
| `@Qualifier` | core | pick a bean by name/qualifier at the injection point (beats `@Primary`) ✔ | §2.5 |
| `@Primary` | core | default bean when several match ✔ | §2.5 |
| `@Fallback` | core (Framework 6.2) | use this bean only if no other candidate qualifies | §2.5 |
| `@Order` | core | order in `List<T>` injection, aspects, filters, listeners (lower first) ✔ | §2.5 |
| `@Value` | core | inject one value: `${key:default}`, SpEL `#{...}` ✔ | §3.5 |
| `@Scope` | core | `singleton` (default), `prototype`, web scopes; `proxyMode` for scoped proxies | §2.6 |
| `@RequestScope` / `@SessionScope` / `@ApplicationScope` | spring-web | web scopes with a CGLIB scoped proxy | §2.6 |
| `@Lookup` | spring-beans | method injection: returns a fresh prototype on each call | §2.6 |
| `@Lazy` | core | create on first use; on an injection point, inject a lazy proxy (breaks cycles) | §2.10, §2.9 |
| `@PostConstruct` / `@PreDestroy` | `jakarta.annotation` | init / destroy callbacks (not for prototypes' destroy) ✔ | §2.7 |
| `@Profile` | core | register only when a profile expression matches (`"!dev"`, `"prod & eu"`) | §2.11 |
| `@Conditional` | core | register only if a `Condition` matches | §2.11 |
| `@EventListener` | core | receive application events (synchronous by default) | §2.12 |
| `@Proxyable` | core (F7) | choose JDK or CGLIB proxy for one bean | §4.4 |
| `@NullMarked` / `@Nullable` | `org.jspecify.annotations` (F7) | null-safety contract; `@Nullable` injection point = optional | §2.13 |
| `@Inject` | `jakarta.inject` (optional jar) | standard alternative to `@Autowired`; `javax.inject` removed in F7 | §2.4 |

### Spring Boot and configuration

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@SpringBootApplication` | spring-boot-autoconfigure / `spring-boot-starter` | `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` | §1.8 |
| `@EnableAutoConfiguration` | spring-boot-autoconfigure | load `AutoConfiguration.imports` candidates and apply them after your beans | §3.10 |
| `@AutoConfiguration` | spring-boot-autoconfigure | marks an auto-configuration class (`before` / `after` ordering) | §3.13 |
| `@ConfigurationProperties("prefix")` | spring-boot | bind a group of properties to a typed object (record ideal) | §3.6 |
| `@ConfigurationPropertiesScan` / `@EnableConfigurationProperties` | spring-boot | register `@ConfigurationProperties` classes | §3.6 |
| `@DefaultValue` | spring-boot | default for a constructor-bound property ✔ | §3.6 |
| `@ConstructorBinding` | spring-boot | pick the binding constructor when there are several | §3.6 |
| `@DurationUnit` / `@DataSizeUnit` | spring-boot | unit for plain numbers (default ms / bytes) ✔ | §3.6 |
| `@Validated` (on properties) | core + `spring-boot-starter-validation` | validate bound properties at startup | §3.6 |
| `@ConditionalOnClass` / `OnMissingClass` | spring-boot-autoconfigure | class present / absent on the classpath | §3.10 |
| `@ConditionalOnBean` / `OnMissingBean` | spring-boot-autoconfigure | bean present / absent (reliable in auto-configurations only) | §3.10 |
| `@ConditionalOnProperty` / `OnBooleanProperty` | spring-boot-autoconfigure | property value / boolean switch (`OnBooleanProperty` since Boot 3.5) | §3.10 |
| `@ConditionalOnWebApplication` / `OnThreading` / `OnCloudPlatform` / ... | spring-boot-autoconfigure | web type, virtual vs platform threads, platform | §3.10 |
| `@PropertySource` | core | add a `.properties` file (low precedence, no YAML) | §3.9 |
| `@EntityScan` | spring-boot-persistence (Boot 4 package) | where to find JPA entities outside the main package | §17.13 |

### AOP and resilience

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@Aspect` (+ `@Component`) | aspectjweaver / `spring-boot-starter-aspectj` (Boot 4) | a class of pointcuts and advice; must also be a bean | §4.2 |
| `@Pointcut` | aspectjweaver | named, reusable pointcut expression | §4.3 |
| `@Before` / `@AfterReturning` / `@AfterThrowing` / `@After` / `@Around` | aspectjweaver | the five advice types; only `@Around` controls `proceed()` | §4.2 |
| `@EnableAspectJAutoProxy` | core | switch on `@Aspect` support (Boot does it: `spring.aop.auto=true`) | §4.2 |
| `@Retryable` | core `org.springframework.resilience` (F7) | retry on exception; `maxRetries` 3, `delay` 1000 ms, `multiplier` 1.0 ✔ — **needs `@EnableResilientMethods`** | §4.7 |
| `@ConcurrencyLimit` | core (F7) | bulkhead: max concurrent calls, policy `BLOCK` or `REJECT` — needs `@EnableResilientMethods` | §4.7 |
| `@EnableResilientMethods` | core (F7) | turns on `@Retryable` and `@ConcurrencyLimit` (no auto-configuration) | §4.7 |
| `@CircuitBreaker` / `@Retry` / `@RateLimiter` / `@Bulkhead` / `@TimeLimiter` | `resilience4j-spring-boot4` | Resilience4j patterns with `fallbackMethod` | §16.8 |

### Web: Spring MVC, WebFlux, REST

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@RequestMapping` | spring-web | class-level prefix / shared mapping attributes | §5.3 |
| `@GetMapping` / `@PostMapping` / `@PutMapping` / `@PatchMapping` / `@DeleteMapping` | spring-web | map an HTTP method + path; `version = "1.1+"` (F7) | §5.3, §5.9 |
| `@PathVariable` | spring-web | URL segment `{id}` | §5.4 |
| `@RequestParam` | spring-web | query parameter / form field (`required = true` by default) | §5.4 |
| `@RequestHeader` / `@CookieValue` | spring-web | header / cookie value | §5.4 |
| `@RequestBody` | spring-web | body converted by an `HttpMessageConverter` (Jackson 3) | §5.4 |
| `@RequestPart` | spring-web | part of a `multipart/form-data` request (`MultipartFile`) | §5.10 |
| `@ModelAttribute` | spring-web | bind query params/form fields to an object | §5.4 |
| `@ResponseStatus` | spring-web | fixed status for a handler or an exception class | §5.4, §5.7 |
| `@ResponseBody` | spring-web | write the return value as the body | §5.3 |
| `@ExceptionHandler` | spring-web | handle an exception type (in a controller or an advice) | §5.7 |
| `@ControllerAdvice` / `@RestControllerAdvice` | spring-web | global exception handling (return `ProblemDetail`) | §5.7 |
| `@Valid` | `jakarta.validation` + `-validation` starter | cascade validation into a body / nested object | §5.6 |
| `@Validated` | core | validation **groups**; on a class: method validation via proxy | §5.6 |
| `@CrossOrigin` | spring-web | per-controller CORS rules | §5.10 |
| `@JsonProperty` / `@JsonIgnore` | `com.fasterxml.jackson.annotation` | JSON names / hidden fields (package unchanged in Jackson 3) | §5.5 |
| `@JacksonComponent` / `@JacksonMixin` | spring-boot-jackson (Boot 4) | register (de)serializers / mix-ins (were `@JsonComponent` / `@JsonMixin`) | §5.5 |
| `@EnableWebMvc` | spring-webmvc | full manual MVC control — **switches Boot's MVC auto-configuration off** | §3.12 |

### HTTP clients

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@HttpExchange` | spring-web | HTTP service interface: shared URL prefix, headers, version | §6.6 |
| `@GetExchange` / `@PostExchange` / `@PutExchange` / `@PatchExchange` / `@DeleteExchange` | spring-web | one HTTP call per interface method | §6.6 |
| `@ImportHttpServices(group, types)` | spring-web (F7) + `spring-boot-starter-restclient` | register HTTP interface proxies per group, configured by `spring.http.serviceclient.<group>.*` | §6.6 |
| `@ClientRegistrationId` | spring-security-oauth2-client | which OAuth2 registration's token to add (type-level since S7) | §11.10 |
| `@LoadBalanced` | spring-cloud-commons | resolve `http://service-id` through Spring Cloud LoadBalancer | §16.5 |
| `@FeignClient` / `@EnableFeignClients` | spring-cloud-openfeign (maintenance mode) | declarative client, Feign style | §16.6 |

### Data access, transactions, JPA and Spring Data

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@Transactional` | spring-tx | declarative transaction (see §19.6) ✔ | §7.8 |
| `@EnableTransactionManagement` | spring-tx | switch on `@Transactional` (Boot does it); `rollbackOn = ALL_EXCEPTIONS` (F6.2) | §7.10 |
| `@TransactionalEventListener` | spring-tx | run a listener at `AFTER_COMMIT` (default), `BEFORE_COMMIT`, `AFTER_ROLLBACK`, `AFTER_COMPLETION` | §7.15 |
| `@Entity` / `@Table` / `@Column` | `jakarta.persistence` / `-data-jpa` | map a class / table / column | §8.3 |
| `@Id` / `@GeneratedValue` | `jakarta.persistence` | primary key and strategy (`IDENTITY`, `SEQUENCE`, `UUID`, `AUTO`) | §8.3 |
| `@ManyToOne` / `@OneToMany` / `@OneToOne` / `@ManyToMany` | `jakarta.persistence` | relationships; to-one EAGER, to-many LAZY by default | §8.5 |
| `@JoinColumn` / `mappedBy` | `jakarta.persistence` | owning side (FK) / inverse side | §8.5 |
| `@Enumerated(EnumType.STRING)` / `@Embedded` / `@Embeddable` / `@Transient` | `jakarta.persistence` | enum as text / value object / not stored | §8.3 |
| `@Version` | `jakarta.persistence` (JPA) or `org.springframework.data.annotation` (Data JDBC, Mongo) | optimistic locking column | §8.14, §9.2 |
| `@Query` | spring-data-jpa / -jdbc / ... | JPQL (JPA), SQL (JDBC), `nativeQuery = true` | §8.10 |
| `@Modifying` | spring-data-jpa / -jdbc | UPDATE/DELETE query; `clearAutomatically` | §8.10 |
| `@Param` | spring-data-commons | name a query parameter (optional with `-parameters`) | §8.10 |
| `@EntityGraph(attributePaths = ...)` | spring-data-jpa | fetch associations in the same query (left join) | §8.7 |
| `@Lock(LockModeType.PESSIMISTIC_WRITE)` | spring-data-jpa | `SELECT ... FOR UPDATE` | §8.14 |
| `@BatchSize` | Hibernate | batch-load lazy collections (N+1 fix) | §8.7 |
| `@DynamicUpdate` | Hibernate | UPDATE only changed columns | §8.4 |
| `@EnableJpaAuditing` / `@EnableJdbcAuditing` | spring-data-jpa / -jdbc | switch on auditing | §8.13, §9.2 |
| `@CreatedDate` / `@LastModifiedDate` / `@CreatedBy` / `@LastModifiedBy` | spring-data-commons | audit columns | §8.13 |
| `@MappedCollection(idColumn, keyColumn)` | spring-data-relational | child collection of a Data JDBC aggregate | §9.2 |
| `@RepositoryRestResource` | spring-data-rest | export (or hide) a repository as HAL endpoints | §9.3 |
| `@Document` / `@Indexed` | spring-data-mongodb | MongoDB document / index | §9.5 |
| `@RedisHash(timeToLive = ...)` | spring-data-redis | Redis repository entity with TTL | §9.6 |

### Caching

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@EnableCaching` | core / `spring-boot-starter-cache` | switch on the cache proxy — without it nothing is cached | §9.8 |
| `@Cacheable` | core | return the cached value; run the method only on a miss (`sync`, `condition`, `unless`) | §9.8 |
| `@CachePut` | core | always run, then store the result | §9.8 |
| `@CacheEvict` | core | remove a key (`allEntries = true` to clear) | §9.8 |
| `@Caching` / `@CacheConfig` | core | several cache ops on one method / class defaults | §9.8 |

### Security

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@EnableWebSecurity` | spring-security-config | web security (Boot applies it for you) | §10.3 |
| `@EnableMethodSecurity` | spring-security-config | switch on method security annotations | §10.7 |
| `@PreAuthorize` / `@PostAuthorize` | spring-security-core | SpEL rule before / after the call (`returnObject`) | §10.7 |
| `@PreFilter` / `@PostFilter` | spring-security-core | filter a collection argument / result | §10.7 |
| `@Secured` / `@RolesAllowed` | spring-security-core / `jakarta.annotation.security` | legacy role checks (`securedEnabled`, `jsr250Enabled`) | §10.7 |
| `@AuthenticationPrincipal` | spring-security-core | inject the principal (`UserDetails`, `Jwt`, `OidcUser`) | §10.4 |
| `@EnableMultiFactorAuthentication` | spring-security-config (S7) | require several `FACTOR_*` authorities | §10.4 |

### Messaging, async and scheduling

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@EnableAsync` | core | switch on `@Async` (Boot configures the executor but does **not** enable it) | §14.4 |
| `@Async("executor")` | core | run on an executor; return `void` or `CompletableFuture` | §14.4 |
| `@EnableScheduling` | core | switch on `@Scheduled` | §14.7 |
| `@Scheduled(fixedRate / fixedDelay / cron)` | core | timer jobs; cron has **six** fields ✔ | §14.7 |
| `@KafkaListener(topics, groupId, concurrency)` | spring-kafka / `spring-boot-starter-kafka` | Kafka consumer method | §14.12 |
| `@RetryableTopic` / `@DltHandler` | spring-kafka | non-blocking retries via retry topics; dead-letter handler | §14.15 |
| `@RabbitListener` | spring-rabbit / `spring-boot-starter-amqp` | RabbitMQ consumer method | §14.18 |
| `@JmsListener` | spring-jms | JMS consumer method | §14.18 |
| `@SchedulerLock` | ShedLock (third party) | run a `@Scheduled` job on one instance only | §14.7 |

**Observability, cloud and AI**

| Annotation | Module / starter | Purpose | § |
|---|---|---|---|
| `@Observed` | micrometer-observation (+ AspectJ starter) | timer + span around a method; needs `management.observations.annotations.enabled=true` | §15.6 |
| `@Timed` / `@Counted` / `@NewSpan` | Micrometer | metric / span annotations (same switch as `@Observed`) | §15.6 |
| `@EnableConfigServer` | spring-cloud-config-server | make the app a Config Server | §16.3 |
| `@RefreshScope` | spring-cloud-context | re-create the bean after a config refresh | §16.3 |
| `@EnableEurekaServer` | spring-cloud-netflix-eureka-server | run a Eureka registry | §16.4 |
| `@Tool` / `@ToolParam` | spring-ai-model (Spring AI 2.0) | expose a method to the model for tool calling | §17.8 |
| `@McpTool` / `@McpToolParam` | Spring AI 2.0 MCP (`org.springframework.ai.mcp.annotation`) | expose a tool from an MCP server | §17.10 |

Testing annotations have their own sheet: §19.9.

---

## 19.4 Starters in Boot 4.1

A **starter** is a code-free dependency that brings one capability's libraries plus the Boot module holding its auto-configuration (§1.7). Boot 4 made them **modular**: one starter per technology, and a matching `-test` starter that brings the test slice for it and already includes `spring-boot-starter-test` (§3.14, §12.3). All names below are managed by the 4.1.1 BOM — never add a `<version>`.

**The starters you will use most**

| Capability | Starter | Test starter (slice, test clients) | § |
|---|---|---|---|
| Core only (no web) | `spring-boot-starter` | `spring-boot-starter-test` | §1.7 |
| Servlet web / REST (Spring MVC + Tomcat + Jackson 3) | `spring-boot-starter-webmvc` | `spring-boot-starter-webmvc-test` (`@WebMvcTest`, MockMvc, `RestTestClient`) | §5.2 |
| Reactive web (WebFlux + Reactor Netty) | `spring-boot-starter-webflux` | `spring-boot-starter-webflux-test` (`@WebFluxTest`, `WebTestClient`) | §13.10 |
| Blocking HTTP client (`RestClient.Builder`, HTTP service groups) | `spring-boot-starter-restclient` | `spring-boot-starter-restclient-test` (`@RestClientTest`) | §6.1 |
| Reactive HTTP client (`WebClient.Builder`) | `spring-boot-starter-webclient` | `spring-boot-starter-webclient-test` | §13.13 |
| JSON (Jackson 3) | `spring-boot-starter-jackson` | `spring-boot-starter-jackson-test` (`@JsonTest`) | §5.5 |
| Bean Validation (Hibernate Validator) | `spring-boot-starter-validation` | `spring-boot-starter-validation-test` | §5.6 |
| AOP with `@Aspect` | `spring-boot-starter-aspectj` | `spring-boot-starter-aspectj-test` | §4.2 |
| JDBC (`DataSource`, HikariCP, `JdbcClient`) | `spring-boot-starter-jdbc` | `spring-boot-starter-jdbc-test` (`@JdbcTest`) | §7.2 |
| JPA (Hibernate + Spring Data JPA) | `spring-boot-starter-data-jpa` | `spring-boot-starter-data-jpa-test` (`@DataJpaTest`) | §8.2 |
| Spring Data JDBC | `spring-boot-starter-data-jdbc` | `spring-boot-starter-data-jdbc-test` (`@DataJdbcTest`) | §9.2 |
| R2DBC | `spring-boot-starter-data-r2dbc` | `spring-boot-starter-data-r2dbc-test` (`@DataR2dbcTest`) | §13.14 |
| MongoDB / Redis | `spring-boot-starter-data-mongodb` / `-data-redis` | `...-data-mongodb-test` / `...-data-redis-test` | §9.5, §9.6 |
| Spring Data REST | `spring-boot-starter-data-rest` | `spring-boot-starter-data-rest-test` | §9.3 |
| Schema migrations | `spring-boot-starter-flyway` / `-liquibase` | `...-flyway-test` / `...-liquibase-test` | §8.16, §9.4 |
| Cache abstraction | `spring-boot-starter-cache` (+ `caffeine`) | `spring-boot-starter-cache-test` | §9.8 |
| Security | `spring-boot-starter-security` | `spring-boot-starter-security-test` | §10.1 |
| OAuth2 resource server / client / authorization server | `spring-boot-starter-security-oauth2-resource-server` / `-client` / `-authorization-server` | `...-test` for each | §11.3, §11.9, §11.14 |
| Kafka | `spring-boot-starter-kafka` (+ `-jackson` for JSON) | `spring-boot-starter-kafka-test` (`@EmbeddedKafka`) | §14.10 |
| RabbitMQ | `spring-boot-starter-amqp` | `spring-boot-starter-amqp-test` | §14.18 |
| Actuator | `spring-boot-starter-actuator` | `spring-boot-starter-actuator-test` | §15.2 |
| Tracing + OTLP export | `spring-boot-starter-opentelemetry` (or `-zipkin` for Brave) | `spring-boot-starter-opentelemetry-test` (`@AutoConfigureTracing`) | §15.7 |
| Metrics only | `spring-boot-starter-micrometer-metrics` (+ a registry, e.g. `micrometer-registry-prometheus`) | `...-micrometer-metrics-test` (`@AutoConfigureMetrics`) | §15.5 |
| HATEOAS | `spring-boot-starter-hateoas` | `...-hateoas-test` | §5.12 |
| Testcontainers support | `spring-boot-testcontainers` (module, test scope) | — | §12.14 |
| Also available | `-websocket`, `-graphql`, `-grpc-server` / `-grpc-client` (Boot 4.1), `-batch`, `-batch-jdbc`, `-integration`, `-quartz`, `-mail`, `-jms`, `-artemis`, `-pulsar`, `-thymeleaf`, `-session-data-redis`, `-session-jdbc`, `-jooq`, `-jetty` | most have a `-test` companion | §1.3 |

### Renamed, deprecated and removed starters

| Boot 3 name | Boot 4.1 status | Use instead | § |
|---|---|---|---|
| `spring-boot-starter-web` | **deprecated** alias (still in the BOM) | `spring-boot-starter-webmvc` | §1.7 |
| `spring-boot-starter-web-services` | **deprecated** alias | `spring-boot-starter-webservices` | §17.13 |
| `spring-boot-starter-oauth2-resource-server` / `-oauth2-client` / `-oauth2-authorization-server` | **deprecated** aliases | `spring-boot-starter-security-oauth2-...` | §11.3 |
| `spring-boot-starter-aop` | **removed** | `spring-boot-starter-aspectj` | §4.2 |
| `spring-boot-starter-undertow` | **removed** (Undertow lacks Servlet 6.1) | Tomcat (default) or `spring-boot-starter-jetty` | §17.13 |
| `spring-boot-starter-json` | still available | `spring-boot-starter-jackson` is the Boot 4 name | §1.7 |
| `flyway-core` / `liquibase-core` alone | no longer auto-configured | `spring-boot-starter-flyway` / `-liquibase` | §8.16 |
| `spring-kafka` alone | still works as a library, but use the starter | `spring-boot-starter-kafka` | §14.10 |
| part of `-web` / `-webflux` in Boot 3 | split out | `spring-boot-starter-restclient`, `spring-boot-starter-webclient` | §6.1, §13.10 |
| `spring-boot-starter-batch` (with JDBC metadata) | batch now runs without a database by default | `spring-boot-starter-batch-jdbc` | §17.13 |
| (migration aid) | new | `spring-boot-starter-classic`, `spring-boot-starter-test-classic`: the old "everything" layout while you migrate | §3.14 |

Third-party starters are named `<name>-spring-boot-starter` (`mybatis-spring-boot-starter`); the `spring-boot-*` prefix is reserved (§3.13).

---

## 19.5 Key properties cheat sheet

Defaults are the Boot 4.1.1 metadata values; "—" means *no default: the feature is off, or the value comes from somewhere else*. Precedence (which source wins) and relaxed binding are at the end of this section.

### Core, profiles and startup

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.application.name` | — | name in logs, traces, Config Server, metrics | §1.9 |
| `spring.profiles.active` / `.default` / `.include` | — / `default` / — | activate profiles (not allowed inside profile-specific files) | §3.7 |
| `spring.profiles.group.<name>` | — | one name activates several profiles | §3.7 |
| `spring.config.import` | — | extra files, `optional:`, `configtree:`, `configserver:` | §3.8 |
| `spring.config.activate.on-profile` | — | make a document of a multi-document file conditional | §3.2 |
| `spring.autoconfigure.exclude` | — | switch one auto-configuration off | §3.12 |
| `debug` | `false` | print the condition evaluation report | §3.11 |
| `spring.main.lazy-initialization` | `false` | make every bean lazy | §2.10 |
| `spring.main.allow-circular-references` | `false` | Boot rejects field/setter cycles (since 2.6) | §2.9 |
| `spring.main.allow-bean-definition-overriding` | `false` | two beans with one name → `BeanDefinitionOverrideException` | §2.3 |
| `spring.main.web-application-type` | detected | `servlet`, `reactive`, `none` (MVC wins if both starters present) | §13.10 |
| `spring.main.keep-alive` | `false` | keep the JVM alive when only virtual (daemon) threads run | §14.5 |
| `spring.aop.proxy-target-class` | `true` | Boot uses CGLIB proxies for everything | §4.4 |
| `spring.aop.auto` | `true` | `@EnableAspectJAutoProxy` applied by Boot | §4.2 |
| `spring.lifecycle.timeout-per-shutdown-phase` | `30s` ✔ | how long graceful shutdown waits | §15.9 |
| `spring.devtools.restart.enabled` | `true` | DevTools restart (off for `java -jar` and tests) | §1.10 |
| `spring.devtools.livereload.enabled` | `false` | LiveReload — off in Boot 4.0, deprecated in 4.1 | §1.10 |

### Web server, MVC and JSON

| Property | Default | What it does | § |
|---|---|---|---|
| `server.port` | `8080` | HTTP port; `0` = random free port | §1.9 |
| `server.shutdown` | `graceful` | `immediate` turns graceful shutdown off (default since Boot 3.4) | §15.9 |
| `server.servlet.context-path` | – | URL prefix of the whole app | §5.2 |
| `server.forward-headers-strategy` | – | `framework` honours `X-Forwarded-*` behind a proxy | §5.4 |
| `server.servlet.session.timeout` | `30m` | HTTP session timeout | §10.10 |
| `server.servlet.session.cookie.secure` / `.http-only` / `.same-site` | – | harden the session cookie | §10.10 |
| `spring.web.error.include-stacktrace` | `never` | stack traces in Boot's error body (Boot 4 name; `server.error.*` no longer bound) | §15.12 |
| `spring.mvc.problemdetails.enabled` | `false` | RFC 9457 bodies for MVC exceptions without your own advice | §5.7 |
| `spring.webflux.problemdetails.enabled` | `false` | the same for WebFlux | §13.10 |
| `spring.mvc.contentnegotiation.favor-parameter` | `false` | allow `?format=csv` | §5.8 |
| `spring.servlet.multipart.max-file-size` / `max-request-size` | `1MB` / `10MB` | upload limits (413 when exceeded) | §5.10 |
| `spring.jackson.default-property-inclusion` | – | e.g. `non_null` | §5.5 |
| `spring.jackson.serialization.*` / `deserialization.*` | – | Jackson feature switches (`indent-output`...) | §5.5 |
| `spring.jackson.json.read.*` / `spring.jackson.json.write.*` | – | JSON parser/generator features (Boot 4; were `spring.jackson.parser/generator.*`) | §5.5 |
| `spring.jackson.use-jackson2-defaults` | `false` | restore Jackson 2 defaults while migrating | §17.13 |
| `spring.threads.virtual.enabled` | `false` | virtual threads for Tomcat, `@Async`, `@Scheduled`, listeners (Java 21+) ✔ | §15.10 |

Jackson 3 defaults under Boot 4 (✔ proved): `LocalDate` written as `"2026-09-28"`, unknown JSON properties ignored, `JacksonException` is unchecked (§5.5).

### API versioning (F7 / Boot 4)

| Property | Meaning | § |
|---|---|---|
| `spring.mvc.apiversion.use.header` | read the version from this header (e.g. `X-API-Version`) | §5.9 |
| `spring.mvc.apiversion.use.query-parameter` / `.use.path-segment` / `.use.media-type-parameter` | other places to read it | §5.9 |
| `spring.mvc.apiversion.default` | version assumed when the request has none | §5.9 |
| `spring.mvc.apiversion.required` | reject requests without a version (400) | §5.9 |
| `spring.mvc.apiversion.supported` / `.detect-supported` | extra supported versions / detect them from mappings | §5.9 |
| `spring.webflux.apiversion.*` | the same for WebFlux | §17.13 |
| `spring.http.serviceclient.<group>.apiversion.insert.header` | client side: send the version in this header | §6.7 |

### HTTP clients

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.http.clients.connect-timeout` / `read-timeout` | – (library default) | global timeouts for auto-configured clients (Boot 4 name; `spring.http.client.*` deprecated) | §6.4 |
| `spring.http.clients.imperative.factory` | detected | `http-components`, `jetty`, `reactor`, `jdk`, `simple` | §6.4 |
| `spring.http.clients.reactive.connector` | detected | the same for `WebClient` | §6.4 |
| `spring.http.clients.redirects` | – | `follow`, `dont-follow`, `follow-when-possible` | §6.4 |
| `spring.http.serviceclient.<group>.base-url` | – | base URL of an `@ImportHttpServices` group (none + Spring Cloud = load-balanced) | §6.6, §16.6 |
| `spring.http.serviceclient.<group>.read-timeout` / `.default-header.<name>` | – | per-group timeout / header | §6.6 |

### DataSource, JDBC, transactions

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.datasource.url` / `.username` / `.password` | – | connection; no URL + embedded DB on the classpath → in-memory DB ✔ | §7.2 |
| `spring.datasource.generate-unique-name` | `true` | random in-memory DB name per context ✔ | §7.2 |
| `spring.datasource.hikari.maximum-pool-size` | `10` (HikariCP) ✔ | most connections | §7.2 |
| `spring.datasource.hikari.minimum-idle` | = max | leave unset: fixed-size pool | §7.2 |
| `spring.datasource.hikari.connection-timeout` | `30000` ms ✔ | wait for a free connection, then fail | §7.2 |
| `spring.datasource.hikari.max-lifetime` / `idle-timeout` / `keepalive-time` | 30 min / 10 min / 2 min | connection recycling | §7.2 |
| `spring.datasource.hikari.leak-detection-threshold` | `0` (off) | log connections held too long | §7.2 |
| `spring.datasource.connection-fetch` | `eager` | `lazy` = no physical connection until the first statement (Boot 4.1) | §7.2 |
| `spring.sql.init.mode` | `embedded` | run `schema.sql` / `data.sql`: `embedded`, `always`, `never` | §7.4 |
| `spring.transaction.default-timeout` | – | default transaction timeout | §7.12 |
| `spring.persistence.exceptiontranslation.enabled` | `true` | `@Repository` exception translation (was `spring.dao.*`) | §7.5 |

### JPA, Hibernate, migrations

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.jpa.hibernate.ddl-auto` | `create-drop` for an embedded DB without a migration tool, else `none` | `validate` in production | §8.2 |
| `spring.jpa.open-in-view` | `true` (+ startup warning) | set `false` for APIs | §8.15 |
| `spring.jpa.show-sql` | `false` | print SQL (`logging.level.org.hibernate.SQL=DEBUG` is better) | §8.2 |
| `spring.jpa.defer-datasource-initialization` | `false` | run `data.sql` after Hibernate created the tables | §7.4 |
| `spring.jpa.properties.hibernate.default_batch_fetch_size` | – | N+1 safety net (e.g. `16`) | §8.7 |
| `spring.jpa.properties.hibernate.jdbc.batch_size` / `order_inserts` | – | batched inserts (needs SEQUENCE ids) | §8.18 |
| `spring.jpa.properties.*` | – | passed to Hibernate unchanged | §8.2 |
| `spring.flyway.enabled` / `.locations` | `true` / `classpath:db/migration` | Flyway (needs `spring-boot-starter-flyway`) | §9.4 |
| `spring.flyway.baseline-on-migrate` / `.clean-disabled` | `false` / `true` | adopt an existing DB / keep `clean` disabled | §9.4 |
| `spring.liquibase.change-log` | `classpath:/db/changelog/db.changelog-master.yaml` | Liquibase master changelog | §9.4 |
| `spring.data.rest.base-path` / `.detection-strategy` | – / `default` | Data REST prefix / `annotated` = only `@RepositoryRestResource` | §9.3 |
| `spring.mongodb.uri` | `mongodb://localhost/test` | MongoDB connection (Boot 4; `spring.data.mongodb.uri` is no longer bound) | §9.5 |
| `spring.data.redis.host` / `.port` | `localhost` / `6379` | Redis connection | §9.6 |
| `spring.r2dbc.url` | – | reactive SQL connection | §13.14 |

### Cache

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.cache.type` | detected (Redis before Caffeine before Simple) | force a provider, or `none` in tests | §9.8 |
| `spring.cache.cache-names` | — | fixed set of caches (typos fail) | §9.8 |
| `spring.cache.caffeine.spec` | — | e.g. `maximumSize=500,expireAfterWrite=10m` | §9.8 |
| `spring.cache.redis.time-to-live` | — (no expiry) | TTL for Redis caches | §9.8 |
| `spring.cache.redis.cache-null-values` / `.key-prefix` | `true` / — | cache `null`s / key prefix (version it) | §9.8 |

### Security

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.security.user.name` / `.password` / `.roles` | `user` / random UUID (logged) / — | the dev-only generated user | §10.1 |
| `spring.security.oauth2.resourceserver.jwt.issuer-uri` | — | discover JWKS, check `iss` | §11.3 |
| `spring.security.oauth2.resourceserver.jwt.jwk-set-uri` / `.public-key-location` | — | keys without discovery | §11.3 |
| `spring.security.oauth2.resourceserver.jwt.audiences` | — | check `aud` (do it!) | §11.3 |
| `spring.security.oauth2.resourceserver.jwt.authorities-claim-name` / `.authority-prefix` / `.principal-claim-name` | — (`scope`/`scp` → `SCOPE_`) | map roles from a claim without code | §11.5 |
| `spring.security.oauth2.resourceserver.opaquetoken.introspection-uri` / `.client-id` / `.client-secret` | — | opaque tokens | §11.7 |
| `spring.security.oauth2.client.registration.<id>.*` / `.provider.<id>.issuer-uri` | — | OAuth2 login / client credentials | §11.9, §11.10 |

**Actuator, health, metrics, tracing, logging**

| Property | Default | What it does | § |
|---|---|---|---|
| `management.endpoints.web.exposure.include` / `.exclude` | `health` | which endpoints are reachable over HTTP (`*` = all) | §15.2 |
| `management.endpoints.web.base-path` | `/actuator` | URL prefix | §15.2 |
| `management.endpoints.access.default` / `management.endpoint.<id>.access` | per endpoint (`shutdown` = `none`) | `none`, `read-only`, `unrestricted` (since 3.4) | §15.2 |
| `management.server.port` | — (same port) | separate management port | §15.2 |
| `management.endpoint.health.show-details` | `never` | `when-authorized`, `always` | §15.3 |
| `management.endpoint.health.probes.enabled` | `true` (Boot 4) | `/actuator/health/liveness` and `/readiness` | §15.3 |
| `management.endpoint.health.group.<name>.include` | — | custom health group | §15.3 |
| `management.endpoint.env.show-values` | `never` | mask values in `env`/`configprops` | §15.13 |
| `management.info.env.enabled` / `.java.enabled` | `false` / `false` | info contributors | §15.4 |
| `management.metrics.tags.<key>` | — | common tag on every meter | §15.5 |
| `management.metrics.distribution.percentiles-histogram.<meter>` | — | histogram buckets for p95/p99 | §15.5 |
| `management.observations.annotations.enabled` | `false` | activate `@Observed`, `@Timed`, `@Counted`, `@NewSpan` | §15.6 |
| `management.observations.enable.<prefix>` | — | switch observation families off | §15.6 |
| `management.tracing.sampling.probability` | `0.1` | share of traces recorded | §15.7 |
| `management.tracing.export.enabled` | `true` | export spans (was `management.tracing.enabled`) | §17.13 |
| `management.opentelemetry.tracing.export.otlp.endpoint` | — | OTLP collector URL (was `management.otlp.tracing.endpoint`) | §15.7 |
| `management.tracing.propagation.type` | W3C | `b3` for legacy Zipkin services | §16.9 |
| `management.tracing.baggage.remote-fields` / `.correlation.fields` | — | baggage in headers / in the MDC | §15.7 |
| `logging.level.<logger>` | `INFO` root | per-logger level (`POST /actuator/loggers/...` at runtime) | §15.2 |
| `logging.structured.format.console` / `.file` | — | `ecs`, `logstash`, `gelf` JSON logs (Boot 3.4+) | §15.8 |
| `logging.pattern.correlation` | Boot's `[traceId-spanId]` | how trace ids appear in text logs | §15.7 |

**Threads, async, scheduling, Kafka, RabbitMQ**

| Property | Default | What it does | § |
|---|---|---|---|
| `spring.task.execution.pool.core-size` | `8` ✔ | `applicationTaskExecutor` threads (`@Async`) | §14.4 |
| `spring.task.execution.pool.queue-capacity` / `.max-size` | unbounded ✔ / unlimited | bound the queue! | §14.4 |
| `spring.task.execution.thread-name-prefix` | `task-` ✔ | thread names | §14.4 |
| `spring.task.execution.mode` | `auto` | `force` keeps Boot's executor next to your own `Executor` beans | §14.4 |
| `spring.task.execution.propagate-context` | `false` | copy tracing/MDC context into `@Async` threads | §14.4 |
| `spring.task.execution.simple.concurrency-limit` | — | cap for the virtual-thread executor | §14.5 |
| `spring.task.execution.shutdown.await-termination` | `false` | finish queued tasks on shutdown | §14.4 |
| `spring.task.scheduling.pool.size` | `1` ✔ | scheduler threads for all `@Scheduled` methods | §14.7 |
| `spring.reactor.context-propagation` | `limited` | `auto` bridges ThreadLocals and the Reactor `Context` | §13.9 |
| `spring.kafka.bootstrap-servers` | `localhost:9092` | brokers (tests: `${spring.embedded.kafka.brokers}`) | §14.10 |
| `spring.kafka.consumer.group-id` / `.auto-offset-reset` | — / Kafka's `latest` | consumer group / where a new group starts | §14.10 |
| `spring.kafka.consumer.isolation-level` | `read-uncommitted` | `read-committed` to hide aborted transactions | §14.16 |
| `spring.kafka.listener.ack-mode` / `.concurrency` | `BATCH` (container default) / 1 | offset commits / consumer threads | §14.12 |
| `spring.kafka.producer.acks` / `.transaction-id-prefix` | Kafka's `all` / — | durability / transactional template | §14.16 |
| `spring.kafka.template.observation-enabled` / `listener.observation-enabled` | `false` / `false` | tracing through Kafka headers | §16.9 |
| `spring.rabbitmq.host` / `.listener.simple.prefetch` | `localhost` / — | broker / unacknowledged messages per consumer | §14.18 |
| `spring.rabbitmq.listener.simple.retry.max-retries` | `3` | retries **after** the first attempt (Boot 4; replaces `max-attempts`) | §14.18 |

**Precedence and relaxed binding (✔ proved)**

Highest wins (§3.3, §12.15):

1. In tests: `@DynamicPropertySource` > `@TestPropertySource` > `@SpringBootTest(properties = ...)`.
2. Command-line arguments `--shop.name=...`
3. `SPRING_APPLICATION_JSON`
4. Java system properties `-Dshop.name=...`
5. OS environment variables `SHOP_NAME=...`
6. `random.*`
7. Profile-specific files `application-{profile}.*` (outside the jar > inside)
8. `application.properties` / `.yaml` (outside the jar > inside; an imported file > its importer)
9. `@PropertySource` files
10. `SpringApplication.setDefaultProperties(...)`

| Rule | Example | § |
|---|---|---|
| env var name = upper case, `.` → `_`, drop `-` | `SHOP_CHECKOUTTIMEOUT=2m` → `shop.checkout-timeout` ✔ | §3.4 |
| list index | `SHOP_CURRENCIES_0=USD` → `shop.currencies[0]` ✔ | §3.4 |
| legacy form (`-` → `_`) is also accepted by the binder | `SHOP_CHECKOUT_TIMEOUT=3m` also binds ✔ — prefer the documented form | §3.4 |
| `Duration` | `45s`, `5m`, `PT1H`; plain `500` = milliseconds ✔ | §3.6 |
| `DataSize` | `10MB` = 10 485 760 bytes; plain `1024` = bytes ✔ | §3.6 |
| `@Value` | `${key}` fails if missing; `${key:default}`; `#{${key:3} + 1}` = SpEL ✔ | §3.5 |

---

## 19.6 Transactions: `@Transactional` at a glance

A Spring transaction = a **proxy** + a `PlatformTransactionManager` + a connection **bound to the current thread** (§7.7). Everything below applies only to calls that go **through the proxy** (§7.13).

### Attributes and their defaults (✔ proved)

| Attribute | Default | Meaning | § |
|---|---|---|---|
| `propagation` | `REQUIRED` | what to do when a transaction already exists (table below) | §7.9 |
| `isolation` | `DEFAULT` = the database's default | only applied when this method **starts** the transaction | §7.11 |
| `readOnly` | `false` | JDBC: read-only connection (PostgreSQL rejects writes); JPA: no dirty checking/flush; can route to a replica | §7.12 |
| `timeout` | `-1` (none) | seconds; enforced on the next SQL statement | §7.12 |
| `rollbackFor` / `noRollbackFor` | – | add / remove exception types from the rollback rule | §7.10 |
| `transactionManager` (`value`) | the primary manager | pick one of several managers | §7.8 |
| `label` | – | free-form labels for custom managers | §7.8 |

Where: on the **service method** that is one use case. Not on controllers (too wide), not only on repositories (too narrow). A class-level `@Transactional(readOnly = true)` + method-level `@Transactional` on writes is the Spring Data pattern (§7.8).

### Propagation (✔ all seven proved)

| Propagation | A transaction exists | None exists | Typical use |
|---|---|---|---|
| `REQUIRED` (default) | **join** it (same connection) | start one | almost everything |
| `REQUIRES_NEW` | **suspend** it, start an independent one on a **second connection**; commits even if the caller later rolls back | start one | audit log that must survive a rollback |
| `NESTED` | **savepoint** inside it; an inner failure rolls back to the savepoint only | start one | "try this part" (JDBC; avoid with JPA) |
| `MANDATORY` | join it | **throw** `IllegalTransactionStateException` | must be called inside a transaction |
| `SUPPORTS` | join it | run **without** a transaction | read helpers that don't care |
| `NOT_SUPPORTED` | **suspend** it, run without | run without | slow non-transactional work inside a flow |
| `NEVER` | **throw** `IllegalTransactionStateException` | run without | guard: never inside a transaction |

The trap ✔: an inner `REQUIRED` method throws, the outer method **catches** the exception and returns normally → the shared transaction was marked **rollback-only** → Spring rolls back at commit and throws `UnexpectedRollbackException` (§7.9).

### Rollback rules (✔ proved)

| Thrown out of the method | Default | `rollbackFor = X` | `noRollbackFor = X` | `rollbackOn = ALL_EXCEPTIONS` (F6.2) |
|---|---|---|---|---|
| `RuntimeException` / `Error` | **rollback** | rollback | commit if it is an X | rollback |
| checked `Exception` | **commit** (!) | rollback if it is an X | commit | rollback |
| caught **inside** the method | commit – the proxy never sees it | commit | commit | commit |

Global switch: `@EnableTransactionManagement(proxyTargetClass = true, rollbackOn = RollbackOn.ALL_EXCEPTIONS)` (§7.10). To mark a transaction without throwing: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` or `status.setRollbackOnly()` in a `TransactionTemplate`.

### Isolation levels

| Isolation | Prevents | Notes |
|---|---|---|
| `DEFAULT` | whatever the database default prevents | PostgreSQL, Oracle, SQL Server: READ COMMITTED; MySQL InnoDB: REPEATABLE READ |
| `READ_UNCOMMITTED` | nothing guaranteed (dirty reads possible) | PostgreSQL treats it as READ COMMITTED |
| `READ_COMMITTED` | dirty reads | non-repeatable reads and lost updates still possible (proved in §7.11) |
| `REPEATABLE_READ` | + non-repeatable reads | PostgreSQL: one snapshot; concurrent update → serialization failure ( `40001` ) → retry the whole transaction |
| `SERIALIZABLE` | + phantoms, write skew | expect serialization failures; always retry |

The anomalies themselves: SQL 13.4.

### Transaction managers and helpers

| You use | Manager Boot creates | § |
|---|---|---|
| JDBC ( `spring-boot-starter-jdbc` ) | `JdbcTransactionManager` (a `DataSourceTransactionManager` ) | §7.7 |
| JPA ( `spring-boot-starter-data-jpa` ) | `JpaTransactionManager` (JDBC on the same `DataSource` joins it) | §7.7 |
| R2DBC | `R2dbcTransactionManager` (context-bound, not thread-bound) | §13.14 |
| Kafka with `spring.kafka.producer.transaction-id-prefix` | `KafkaTransactionManager` | §14.16 |
| Programmatic | `TransactionTemplate.execute(status -> ...)` — no proxy, no self-invocation problem | §7.14 |
| After-commit side effects | `@TransactionalEventListener` ( `AFTER_COMMIT` default; skipped without a transaction unless `fallbackExecution = true` ; a transactional listener must use `REQUIRES_NEW` ) | §7.15 |

### "Why is my `@Transactional` not working?"

| Symptom | Cause | § |
|---|---|---|
| partial writes after an exception | self-invocation ( `this.save()` ), `private` method, bean made with `new` | §7.13, §4.5 |
| `NullPointerException` inside a `final` `@Transactional` method | the method ran on the CGLIB proxy instance, whose fields are `null` | §4.5 |
| data committed although the method threw | checked exception without `rollbackFor` , or exception caught inside | §7.10 |
| `UnexpectedRollbackException` | inner `REQUIRED` failure was caught | §7.9 |
| `isolation` / `timeout` ignored | the method joined an existing transaction | §7.11 |
| works in tests, fails in production | the test was `@Transactional` and supplied the transaction | §12.7 |
| `@Async` code doesn't see uncommitted data | transactions are bound to one thread | §7.13 |
| pool exhausted under load | `REQUIRES_NEW` holds two connections; remote calls inside transactions | §7.9, §7.13 |

---

## 19.7 Bean scopes and the lifecycle

### Scopes (✔ singleton and prototype proved)

| Scope | Instances | Destroy callbacks? | Declare with | § |
|---|---|---|---|---|
| `singleton` (default) | one per container, shared by all threads — keep it **stateless** | yes | nothing | §2.6 |
| `prototype` | a new one per `getBean`/injection | **no** — Spring forgets it | `@Scope("prototype")` | §2.6 |
| `request` | one per HTTP request | yes | `@RequestScope` (scoped proxy) | §2.6 |
| `session` | one per HTTP session | yes | `@SessionScope` | §2.6 |
| `application` | one per `ServletContext` | yes | `@ApplicationScope` | §2.6 |
| `websocket` | one per WebSocket session | yes | `@Scope("websocket")` | §2.6 |
| refresh (Spring Cloud) | re-created after a config refresh | yes | `@RefreshScope` | §16.3 |

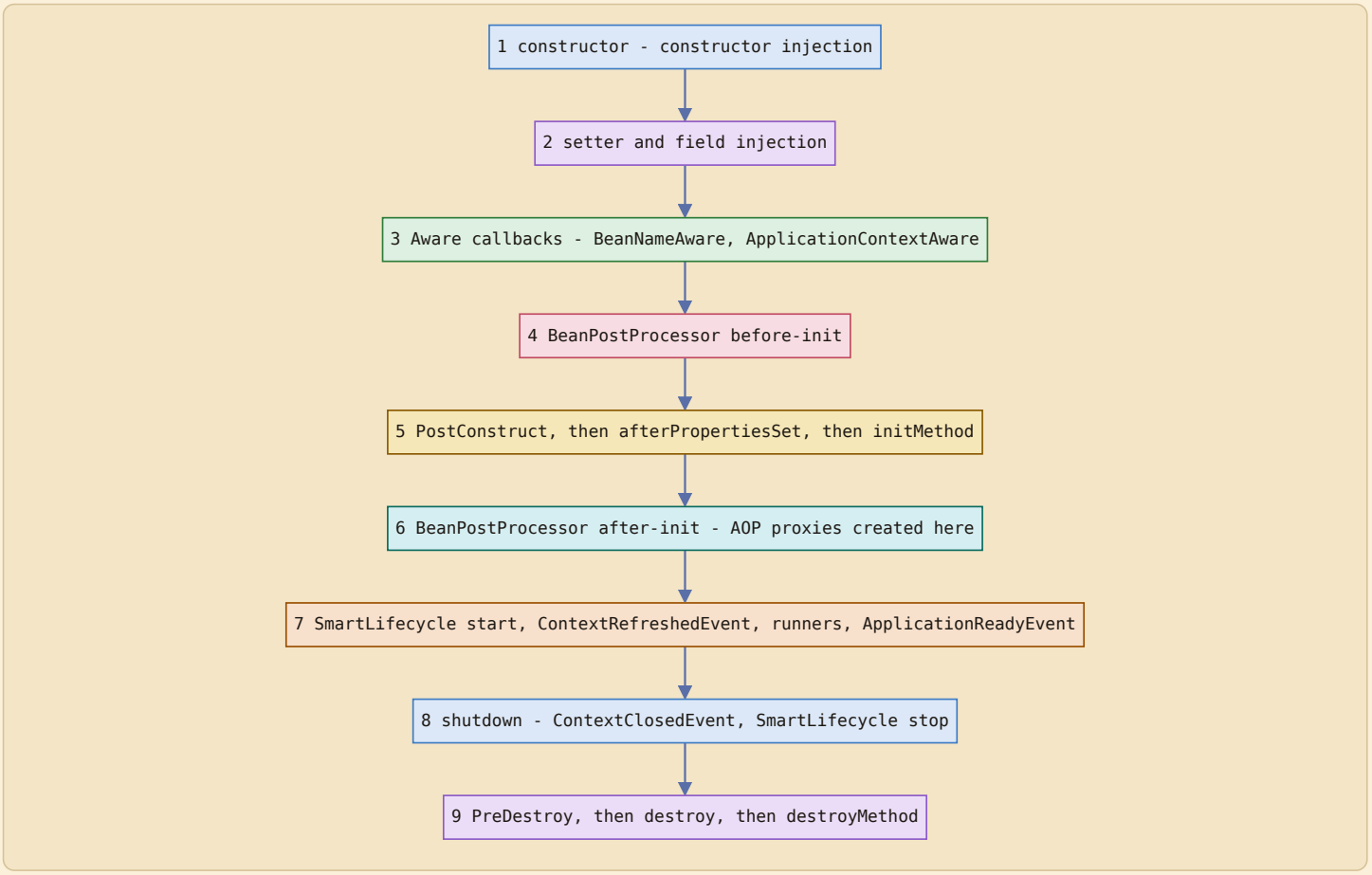
A prototype injected into a singleton is created **once** and stays ✔. For a fresh one each time: `ObjectProvider<T>.getObject()`, a `@Lookup` method, or a scoped proxy (§2.6).

### How Spring picks one bean among several (✔ proved)

1. `@Qualifier("name")` on the injection point
2. `@Primary` bean
3. the only non-`@Fallback` bean (F6.2)
4. a bean whose name equals the parameter name (needs `-parameters`, which the Boot parent sets)
5. otherwise `NoUniqueBeanDefinitionException`

`List<T>` gets all beans ordered by `@Order` (lowest first); `Map<String, T>` is keyed by bean name. Optional dependencies: `Optional<T>`, `ObjectProvider<T>`, `@Nullable`, `@Autowired(required = false)` (§2.5).

### Lifecycle order (✔ proved)




Proved order: init `[constructor, @PostConstruct, afterPropertiesSet, initMethod]`, destroy `[@PreDestroy, destroy(), destroyMethod]`, and no `@PreDestroy` for the prototype.

| Hook | Use for | § |
|---|---|---|
| `@PostConstruct` / `@PreDestroy` | default choice: validate, warm up, release | §2.7 |
| `InitializingBean` / `DisposableBean` | framework code (couples you to Spring) | §2.7 |
| `@Bean(initMethod, destroyMethod)` | third-party classes; a public `close()` / `shutdown()` is inferred | §2.7 |
| `SmartLifecycle` (`getPhase()`) | things that *run*: pollers, listeners (start low phase first, stop it last) | §2.7 |
| `CommandLineRunner` / `ApplicationRunner` | one-off startup work with the arguments | §1.8 |
| `@EventListener(ApplicationReadyEvent.class)` | "after the app is fully started" | §2.7 |
| `BeanFactoryPostProcessor` (declare `static`) | change bean **definitions** before any bean exists | §2.8 |
| `BeanPostProcessor` (declare `static`) | change or wrap every bean **instance** | §2.8 |

Pitfall: a `@Transactional` call from `@PostConstruct` has no transaction — the proxy is created in step 6, after step 5, and the call is on `this` anyway (§2.7).

---

## 19.8 Spring Security 7 DSL snippets

Security 7 accepts only the **lambda DSL**: one lambda per feature, no `.and()`, `authorizeHttpRequests` (not `authorizeRequests`), path matchers built on `PathPattern` (§10.3). The four snippets below are compiled by every build of `SecuritySnippets.java`, so each DSL call exists in Security 7.1.1. Rules are checked **top to bottom, first match wins** (§10.6).

### Form login + HTTP Basic (browser app)

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth                      // first matching rule wins
            .requestMatchers("/", "/public/**", "/login").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")        // checks authority ROLE_ADMIN
            .requestMatchers(HttpMethod.GET, "/reports/**").hasAuthority("report:read")
            .anyRequest().authenticated())                        // or denyAll() to fail closed
        .formLogin(form -> form.loginPage("/login").permitAll()) // omit loginPage for the generated page
        .httpBasic(Customizer.withDefaults())
        .logout(Customizer.withDefaults());                       // POST /logout (+ CSRF token)
    return http.build();
}

@Bean
PasswordEncoder passwordEncoder() {                               // {bcrypt} by default
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

@Bean
static RoleHierarchy roleHierarchy() {                            // admins also pass hasRole("USER")
    return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER");
}
```

Plus a `UserDetailsService` bean (in-memory in the snippet; JPA/JDBC-backed in real apps). Defining it (or any `SecurityFilterChain`) makes Boot's generated user and default chain back off (§10.1).

### JWT resource server (stateless API, first of two chains)

```
@Bean
@Order(1)                                                         // specific chain first
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")                               // which requests this chain owns
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/**").hasAuthority("SCOPE_orders.read")
            .requestMatchers(HttpMethod.POST, "/api/**").hasAuthority("SCOPE_orders.write")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))   // issuer-uri from properties
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(csrf -> csrf.disable());                            // no cookies on this chain
    return http.build();
}
```

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://idp.example.com/realms/shop
spring.security.oauth2.resourceserver.jwt.audiences=orders-api
# optional: roles from a claim, without writing a converter
spring.security.oauth2.resourceserver.jwt.authorities-claim-name=roles
spring.security.oauth2.resourceserver.jwt.authority-prefix=ROLE_
```

Note the last two lines replace the default `scope` → `SCOPE_` mapping; to keep both, write a converter that combines two `JwtGrantedAuthoritiesConverter`s (§11.5). The browser chain (no `securityMatcher`) gets `@Order(2)` — catch-all chains go last (§10.3).

**Method security**

```
@Configuration
@EnableMethodSecurity                                          // @PreAuthorize & co. (proxy-based)
class MethodSecurityConfig { }

public class OrderService {
    @PreAuthorize("hasRole('ADMIN') or #owner == authentication.name")
    public List<Order> ordersOf(String owner) { ... }

    @PostAuthorize("returnObject.owner() == authentication.name")
    public Order find(long id) { ... }

    @PreAuthorize("hasAuthority('order:refund')")
    public String refund(long id) { ... }
}
```

Proxy rules apply: self-invocation and `private` methods are **not** protected (§10.7).

**CORS and CSRF for a cookie-based single-page app**

```
@Bean
SecurityFilterChain spa(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .cors(Customizer.withDefaults())         // uses the CorsConfigurationSource bean below
        .csrf(csrf -> csrf.spa())                 // Security 7: XSRF-TOKEN cookie + X-XSRF-TOKEN header
        .formLogin(Customizer.withDefaults());
    return http.build();
}

@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration api = new CorsConfiguration();
    api.setAllowedOrigins(List.of("https://app.example.com"));   // exact origins, never "*" with credentials
    api.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    api.setAllowedHeaders(List.of("Content-Type", "X-XSRF-TOKEN"));
    api.setAllowCredentials(true);
    api.setMaxAge(3600L);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
}
```

### The rules in one table

| Fact | Detail | § |
|---|---|---|
| `hasRole("ADMIN")` | checks the authority `ROLE_ADMIN` ✔; `hasRole("ROLE_ADMIN")` throws `IllegalArgumentException` ✔ | §10.6 |
| `hasAuthority("x")` | checks the literal string ( `report:read` , `SCOPE_orders.read` ) | §10.6 |
| JWT scopes | `scope` / `scp` claim → `SCOPE_x` authorities | §11.5 |
| 401 vs 403 | anonymous + denied → 401 (or login redirect); authenticated + denied → 403 | §10.6 |
| JWT failures | bad/expired token → 401 `invalid_token` ; missing scope → 403 `insufficient_scope` ; 60 s clock skew | §11.3 |
| CSRF | on for unsafe methods; disable only on chains without cookies/cached Basic | §10.8 |
| CORS | `http.cors(...)` so preflights are answered before authentication | §10.9 |
| sessions | `IF_REQUIRED` (default), `ALWAYS` , `NEVER` , `STATELESS` ; session id changes at login | §10.10 |
| passwords | `DelegatingPasswordEncoder` : `{bcrypt}` by default ✔, reads `{noop}` , `{argon2...}` ; `upgradeEncoding` → re-hash on login ✔ | §10.5 |
| MFA (S7) | every login adds a `FACTOR_*` authority ( `FACTOR_PASSWORD` , `FACTOR_OTT` , `FACTOR_WEBAUTHN` ) | §10.4 |
| removed in S7 | `.and()` , `authorizeRequests` , `AntPathRequestMatcher` / `MvcRequestMatcher` , OAuth2 password grant client support | §17.15 |
| security context | `SecurityContextHolder` is a `ThreadLocal` : use `DelegatingSecurityContext*` executors for `@Async` | §10.2 |

---

## 19.9 Testing annotations and slices

### Whole-application tests

| Annotation / type | Module / test starter | Purpose | § |
|---|---|---|---|
| `@SpringBootTest` | spring-boot-test ( `spring-boot-starter-test` ) | full context; `webEnvironment = MOCK` (default), `RANDOM_PORT` , `DEFINED_PORT` , `NONE` | §12.6 |
| `@AutoConfigureMockMvc` | `-webmvc-test` | `MockMvc` + `MockMvcTester` in a `@SpringBootTest` (no longer automatic in Boot 4) | §12.6 |
| `@AutoConfigureRestTestClient` | spring-boot-resttestclient (in `-webmvc-test` ) | inject `RestTestClient` (F7), bound to MockMvc or the running server | §12.9 |
| `@AutoConfigureWebTestClient` | `-webflux-test` | `WebTestClient` in a `@SpringBootTest` | §13.12 |
| `@AutoConfigureTestRestTemplate` | spring-boot-resttestclient | legacy `TestRestTemplate` | §12.9 |
| `@LocalServerPort` | spring-boot-test | the random port | §1.12 |
| `@TestConfiguration` | spring-boot-test | extra beans; nested static = added, top-level = `@Import` it | §12.6 |
| `@ActiveProfiles` | spring-test | activate profiles (part of the cache key) | §12.15 |
| `@TestPropertySource` / `properties = ...` | spring-test / spring-boot-test | test-only properties | §12.15 |
| `@DynamicPropertySource` | spring-test | runtime values via `Supplier` s; highest precedence | §12.12 |
| `@DirtiesContext` | spring-test | close and evict the cached context (avoid) | §12.11 |
| `@Sql` | spring-test | run SQL scripts before/after a test | §12.16 |
| `@Transactional` (on a test) | spring-test | rolled back after each test — can hide transaction bugs | §12.7, §12.17 |

### Slices

| Slice | Test starter | Loads | Rolled back? | Extras | § |
|---|---|---|---|---|---|
| `@WebMvcTest(Controller.class)` | `spring-boot-starter-webmvc-test` | MVC, Jackson, validation, security filters, your controllers/advice/filters | no | `MockMvcTester`, `MockMvc` | §12.7 |
| `@WebFluxTest` | `spring-boot-starter-webflux-test` | WebFlux, codecs, controllers | no | `WebTestClient` | §13.10 |
| `@DataJpaTest` | `spring-boot-starter-data-jpa-test` | JPA, repositories, embedded DB, Flyway/Liquibase | **yes** | `TestEntityManager`, SQL shown | §12.7 |
| `@JdbcTest` | `spring-boot-starter-jdbc-test` | `DataSource`, `JdbcClient`, `JdbcTemplate` | **yes** | — | §7.6 |
| `@DataJdbcTest` | `spring-boot-starter-data-jdbc-test` | Spring Data JDBC + Flyway | **yes** | — | §9.2 |
| `@DataR2dbcTest` | `spring-boot-starter-data-r2dbc-test` | R2DBC repositories | **no** | — | §13.14 |
| `@JsonTest` | `spring-boot-starter-jackson-test` | Jackson `JsonMapper` with your customisations | no | `JacksonTester` | §12.7 |
| `@RestClientTest(Client.class)` | `spring-boot-starter-restclient-test` | `RestClient.Builder`, Jackson, the named client | no | `MockRestServiceServer` | §6.8 |
| `@DataMongoTest` / `@DataRedisTest` | `...-data-mongodb-test` / `...-data-redis-test` | the store's repositories | no | — | §9.7 |
| `@AutoConfigureTestDatabase(replace = ...)` | `spring-boot-starter-jdbc-test` | `NON_TEST` (default since 3.4) replaces your DB unless it is already a test DB; `NONE` keeps it | — | — | §12.7 |

**Mocks, containers, security helpers**

| Annotation / helper | Module | Purpose | § |
|---|---|---|---|
| `@MockitoBean` | spring-test (F6.2) | Mockito mock **in the context** (replaced removed `@MockBean`); reset after each test | §12.10 |
| `@MockitoSpyBean` | spring-test | spy around the real bean (replaced `@SpyBean`) | §12.10 |
| `@TestBean` | spring-test | override a bean with a static factory method's result | §12.10 |
| `@Mock` + `@ExtendWith(MockitoExtension.class)` | Mockito | plain unit test, no Spring | §12.4 |
| `@Testcontainers` / `@Container` | Testcontainers 2 | start Docker containers for the test | §12.14 |
| `@ServiceConnection` | spring-boot-testcontainers | take connection details from the container (no URL properties) | §12.14 |
| `@EmbeddedKafka` | spring-kafka-test (`-kafka-test`) | in-JVM KRaft broker; sets `spring.embedded.kafka.brokers` | §14.13 |
| `@AutoConfigureMetrics` / `@AutoConfigureTracing` | `-micrometer-metrics-test` / `-opentelemetry-test` | keep metric export / span export on in tests | §15.5, §15.7 |
| `@WithMockUser` / `@WithUserDetails` / `@WithAnonymousUser` | spring-security-test | identity for a test method | §10.12 |
| `user()`, `httpBasic()`, `csrf()`, `formLogin()`, `logout()` | spring-security-test | request post-processors / builders | §10.12 |
| `jwt()`, `opaqueToken()`, `oauth2Login()`, `oidcLogin()` | spring-security-test | token identities — pass your converter to `jwt()` | §11.6 |
| `ApplicationContextRunner` | spring-boot-test | tiny context per test, for auto-configuration and conditions | §3.13 |
| `StepVerifier` (+ `withVirtualTime`) | reactor-test | verify `Mono`/`Flux` signals | §13.5 |
| Awaitility `await().atMost(...)` | awaitility (in `spring-boot-starter-test`) | wait for asynchronous results, never `Thread.sleep` | §14.2 |

**Test clients and the context cache**

| Client | Stack | Needs a server? | § |
|---|---|---|---|
| `MockMvcTester` (F6.2, AssertJ style) | MVC | no | §12.8 |
| `RestTestClient` (F7) | MVC (or any server with `bindToServer`) | optional | §12.9 |
| `WebTestClient` | WebFlux (also MockMvc) | optional | §13.12 |
| `TestRestTemplate` | any | yes | §12.9 |

Context cache: key = the merged configuration (classes, profiles, properties, `@MockitoBean`s, `@DynamicPropertySource`, `@AutoConfigure...`, `webEnvironment`); **max 32** contexts, LRU (`spring.test.context.cache.maxSize`); Framework 7 **pauses** inactive contexts (`spring.test.context.cache.pause`). Keep one shared base class (§12.11).

---

## 19.10 Reactor operators quick table

`Mono` = 0..1 item, `Flux` = 0..N items; both end with complete **or** error, and **nothing runs until someone subscribes** (in WebFlux, the framework does) (§13.3).

| Goal | Operator(s) | Remember | § |
|---|---|---|---|
| create from values | `Mono.just(v)`, `Flux.just(...)`, `Flux.fromIterable(list)`, `Flux.range(1, n)` | `just(expr)` evaluates `expr` **immediately** | §13.3 |
| create lazily | `Mono.fromCallable(...)`, `Mono.defer(() -> ...)`, `Mono.fromFuture(...)` | runs per subscription — retries repeat it | §13.3 |
| empty / error | `Mono.empty()`, `Mono.error(e)` | | §13.3 |
| timers | `Flux.interval(d)`, `Mono.delay(d)`, `delayElements(d)` | run on `Schedulers.parallel()` | §13.3 |
| bridge callbacks | `Flux.create(sink -> ...)`, `Sinks.many()`, `Sinks.one()` | choose an overflow strategy | §13.9 |
| transform (sync) | `map` | must not block or return a publisher | §13.4 |
| async call per item | `flatMap(fn[, concurrency])` | concurrent, **interleaved** output; default concurrency 256 | §13.4 |
| async, one at a time, in order | `concatMap` | sequential (sum of times) | §13.4 |
| async, concurrent, source order | `flatMapSequential` | buffers early results | §13.4 |
| filter | `filter`, `take(n)`, `skip(n)`, `distinct` | | §13.4 |
| combine by position | `Mono.zip(a, b, c)`, `zipWith` | "call 3 services in parallel" (like `Promise.all`) | §13.4 |
| combine in sequence / as they come | `concat`, `concatWith` / `merge`, `mergeWith` | | §13.4 |
| fallback when empty | `switchIfEmpty(...)`, `defaultIfEmpty(v)` | the usual 404 path | §13.4 |
| aggregate | `collectList`, `collectMap`, `reduce`, `count` | return a `Mono` | §13.4 |
| group | `buffer(n)`, `window(n)`, `groupBy` | | §13.4 |
| then | `then()`, `then(mono)` | "do A, then B", ignoring A's values | §13.4 |
| side effects | `doOnNext`, `doOnError`, `doOnSubscribe`, `doFinally`, `log()` | not for business logic | §13.4 |
| errors: fallback | `onErrorReturn(v)`, `onErrorResume(e -> pub)`, `onErrorMap(e -> ex)` | an error **ends** the sequence | §13.6 |
| errors: retry | `retry(n)`, `retryWhen(Retry.backoff(3, Duration.ofSeconds(1)))` | re-subscribes; attempts at 0/1/3/7 s with `jitter(0)` | §13.6 |
| timeout | `timeout(d[, fallback])` | before `retryWhen` = per attempt | §13.6, §13.13 |
| backpressure | `limitRate(n)` (re-requests at 75 %), `onBackpressureDrop/Latest/Buffer(n)/Error` | unbounded buffers risk `OutOfMemoryError` | §13.7 |
| threads | `publishOn(s)` (everything **after** it), `subscribeOn(s)` (the **source**; position irrelevant) | | §13.8 |
| schedulers | `Schedulers.parallel()`, `boundedElastic()` (wrap blocking calls), `single()`, `immediate()` | never block Netty event loops | §13.8 |
| hot / shared | `share()`, `publish().autoConnect()`, `cache()`, `Sinks.many().multicast()` | late subscribers miss earlier items | §13.9 |
| context | `contextWrite(c -> c.put(k, v))`, `Mono.deferContextual(ctx -> ...)` | the reactive `ThreadLocal` | §13.9 |
| leave the reactive world | `block()`, `blockFirst()`, `blockLast()` | tests and `main` only — throws on event-loop threads | §13.3 |
| test | `StepVerifier.create(pub).expectNext(...).verifyComplete()`, `withVirtualTime(() -> ...)` + `thenAwait(d)` | build the publisher **inside** the supplier | §13.5 |

---

## 19.11 Actuator endpoints

Add `spring-boot-starter-actuator`. Two switches decide availability: **access** (`none` / `read-only` / `unrestricted`) and **exposure** (HTTP / JMX). By default only `health` is exposed over HTTP (§15.2).

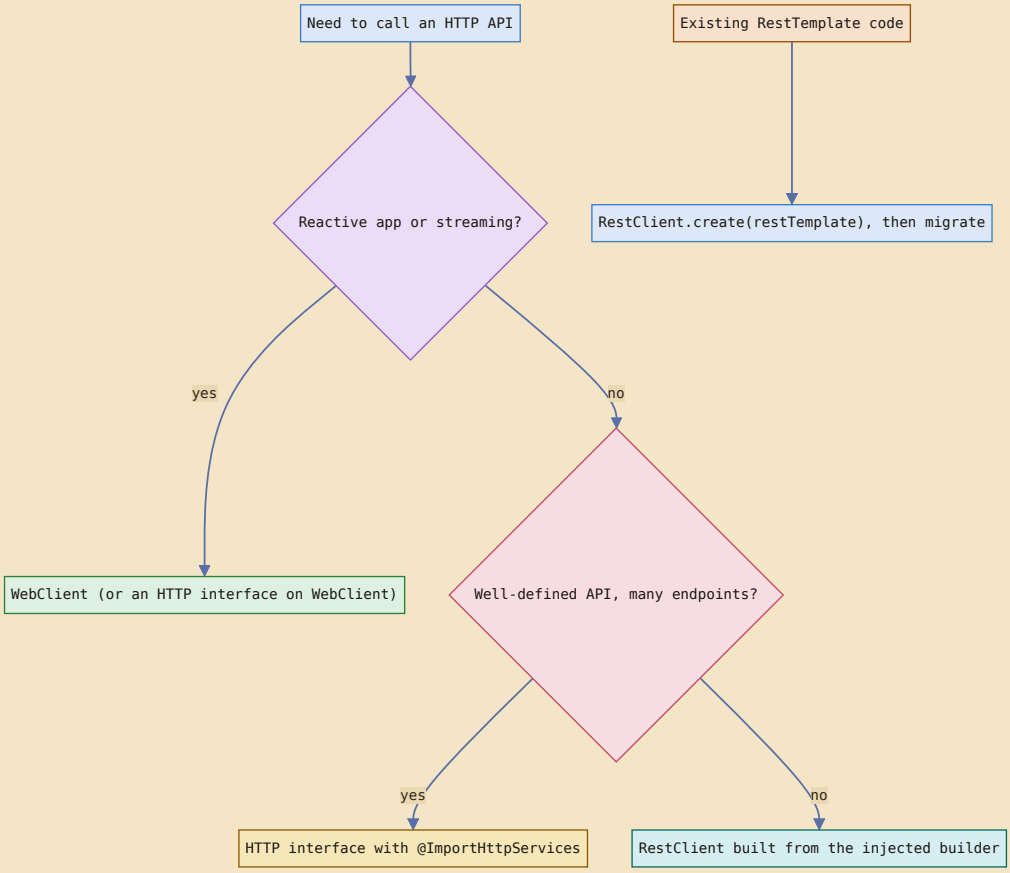
| Endpoint ( `/actuator/<id>` ) | Shows / does | Notes | § |
|---|---|---|---|
| `health` | status `UP` / `DOWN` / `OUT_OF_SERVICE` / `UNKNOWN`; groups `liveness`, `readiness` (on by default in Boot 4) | `DOWN` / `OUT_OF_SERVICE` → HTTP 503; details `never` by default | §15.3 |
| `info` | build, git, java, os, process, `info.*` | contributors switched on by properties | §15.4 |
| `metrics`, `metrics/{name}` | meters as JSON, drill down with `?tag=k:v` | for humans | §15.5 |
| `prometheus` | all meters in Prometheus text format | needs `micrometer-registry-prometheus` | §15.5 |
| `loggers` | view and change log levels at runtime | `POST` with `{"configuredLevel":"DEBUG"}` | §15.2 |
| `env`, `configprops` | property sources, bound `@ConfigurationProperties` | values masked ( `show-values=never` ) | §3.11, §15.2 |
| `conditions` | the auto-configuration report | "why was bean X (not) created?" | §3.11 |
| `beans`, `mappings` | all beans / all request mappings | debugging | §15.2 |
| `threaddump`, `heapdump` | JVM threads / heap dump file | heap dumps can contain secrets | §15.2 |
| `scheduledtasks`, `caches`, `flyway`, `liquibase`, `sessions`, `quartz` | feature-specific views ( `caches` can also clear) | appear when the feature is present | §15.2 |
| `httpexchanges` | recent requests/responses | needs an `HttpExchangeRepository` bean | §15.2 |
| `startup` | startup steps and durations | needs `BufferingApplicationStartup` | §15.2 |
| `sbom`, `auditevents`, `integrationgraph`, `logfile` | software bill of materials, audit events, Spring Integration graph, the log file | when configured | §15.2 |
| `shutdown` | stops the app | access `none` by default | §15.2 |
| `refresh`, `busrefresh` (Spring Cloud) | re-bind config / broadcast a refresh | Spring Cloud Context / Bus | §16.3 |

Secure them with `EndpointRequest.to("health", "info")` / `EndpointRequest.toAnyEndpoint()` (Boot 4 package `org.springframework.boot.security.autoconfigure.actuate.web.servlet`), ideally on a separate `management.server.port` (§15.2).

---

## 19.12 Which tool for which job

**HTTP client: `RestClient` vs `WebClient` vs HTTP interface**




| | `RestClient` | `WebClient` | **HTTP interface ( `@HttpExchange` )** | `RestTemplate` |
|---|---|---|---|---|
| Style | fluent, blocking | fluent, non-blocking ( `Mono` / `Flux` ) | declarative Java interface → proxy | template methods |
| Since / status | F6.1, recommended for blocking code | F5.0, recommended for reactive | F6.0; Boot 4 groups via `@ImportHttpServices` | deprecation announced (F7.1 `@Deprecated` , removal in 8.0) |
| Starter (Boot 4) | `spring-boot-starter-restclient` | `spring-boot-starter-webclient` | `-restclient` (or `-webclient` with `clientType = WEB_CLIENT` ) | `-restclient` |
| Pick when | hand-written calls, custom per-call logic | WebFlux apps, streaming, heavy async composition | typed clients to other services, least boilerplate | never for new code |
| Watch out | inject `RestClient.Builder` (Boot converters, timeouts, tracing); `exchange()` skips status handlers | don't `block()` on event loops | wrap for `Optional` /domain exceptions | — |
| § | §6.2 | §13.13 | §6.6 | §6.9 |

### Web stack: Spring MVC + virtual threads vs WebFlux

| | MVC + `spring.threads.virtual.enabled=true` | WebFlux |
|---|---|---|
| Code style | plain imperative, blocking libraries OK | `Mono` / `Flux` pipelines, non-blocking everything |
| Measured (1,000 clients, 100 ms I/O) | **7,942 req/s**, p50 108 ms, 33 platform threads | **8,058 req/s**, p50 110 ms |
| Same test, MVC on 200 platform threads | 1,909 req/s, p50 517 ms | WebFlux + one `Thread.sleep`: **97 req/s** |
| Data access | JDBC, JPA | R2DBC, reactive Mongo/Redis |
| Streaming, backpressure end to end (SSE, WebSockets) | possible (`SseEmitter`), each stream holds a request | natural (`Flux`) |
| Debugging, `ThreadLocal` libraries | easy | needs Reactor `Context`, context propagation |
| **Choose for** | typical CRUD/REST services (the default) | streaming, gateways, all-non-blocking pipelines |
| § | §13.15, §15.10 | §13.15 |

### Data access: JPA vs `JdbcClient` vs Spring Data JDBC

| | Spring Data JPA (Hibernate) | `JdbcClient` / `JdbcTemplate` | Spring Data JDBC |
|---|---|---|---|
| Model | rich object graph, relationships both ways | your own SQL, rows → records | DDD aggregates: root + owned children |
| Magic | persistence context, lazy loading, dirty checking, first-level cache | none | none: `save` writes, loading reads the whole aggregate |
| SQL you can predict | not always (N+1, flush timing) | yes | yes (children are deleted + re-inserted on save) |
| Schema | `ddl-auto` possible (use Flyway) | you write it | you write it (Flyway) |
| Typical pitfalls | N+1, `LazyInitializationException`, OSIV | SQL injection by concatenation, `.single()` on 0 rows | assigned ids without `@Version` → silent `UPDATE` of 0 rows |
| **Choose for** | domain models with relationships, team knows JPA | reports, bulk work, full SQL control | clear aggregates, predictable SQL |
| § | §8.1 | §7.3 | §9.2 |

Also: R2DBC for non-blocking SQL (§13.14), MongoDB for flexible documents (§9.5), Redis for counters/sessions/shared cache (§9.6); the overview table is §9.9.

### Cache: Caffeine vs Redis

| | Caffeine (local) | Redis (shared) |
|---|---|---|
| Read cost | nanoseconds, no serialization | network hop + (de)serialization |
| Consistency across instances | each instance has its own copy (stale until TTL) | one copy for all |
| Survives restarts | no | yes (with Redis persistence) |
| Stores | object **references** — cache immutable values | serialized copies (choose a JSON serializer) |
| Stampede protection | `@Cacheable(sync = true)` per key per JVM | same annotation, still per JVM; distributed lock for cluster-wide |
| TTL / size | `expireAfterWrite`, `maximumSize` (`spring.cache.caffeine.spec`) | `spring.cache.redis.time-to-live` / `RedisCacheConfiguration` |
| **Choose for** | small, hot, rarely changing reference data | data shared between instances, sessions, expensive results |
| § | §9.8 | §9.8 |

### Messaging: Kafka vs RabbitMQ

| | Kafka | RabbitMQ |
|---|---|---|
| Model | partitioned, replicated **log**; consumers track offsets | **queue** broker: exchanges → bindings → queues; acked messages are deleted |
| Replay / many independent readers | yes: each consumer group has its own offset | no: each reader needs its own queue |
| Ordering | per partition (use the aggregate id as key) | per queue, weakened by competing consumers |
| Routing | topic + partition only | rich: direct, topic, fanout, headers |
| Per-message TTL, priority, delay | no | yes (delay via plugin) |
| Spring | `KafkaTemplate`, `@KafkaListener`, `DefaultErrorHandler` → `<topic>-dlt`, `@RetryableTopic` | `RabbitTemplate`, `@RabbitListener`, DLX, publisher confirms |
| Test without Docker | `@EmbeddedKafka` | no embedded broker (Testcontainers) |
| **Choose for** | event streaming, event sourcing, high throughput | task distribution, complex routing, request/reply |
| § | §14.9 | §14.18 |

### Resilience: Resilience4j vs Framework 7 `@Retryable`

| Need | Framework 7 core | Resilience4j | Spring Retry |
|---|---|---|---|
| Retry with back-off + jitter | `@Retryable` (`maxRetries` = retries **after** the first call ✔), `RetryTemplate` | `@Retry` (`max-attempts` counts the first call) | yes (no longer managed by Boot 4) |
| Bulkhead | `@ConcurrencyLimit` (`BLOCK` / `REJECT`) | `@Bulkhead` (semaphore / thread pool) | no |
| Circuit breaker | **no** | `@CircuitBreaker` (CLOSED → OPEN → HALF_OPEN) | basic |
| Rate limiter / time limiter | no | `@RateLimiter`, `@TimeLimiter` | no |
| Fallback methods | no (catch at the caller) | `fallbackMethod` | `@Recover` |
| Metrics, health | `MethodRetryEvent`, `RetryListener` | Micrometer meters, health indicators | listeners |
| Switch | `@EnableResilientMethods` | `resilience4j-spring-boot4` + properties | `@EnableRetry` |
| **Choose for** | simple retries and concurrency caps, no extra dependency | circuit breaking, rate limits, fallbacks | legacy code only |
| § | §4.7 | §16.8 | §4.7 |

Rules for any retry: only transient failures and idempotent operations, exponential back-off with jitter, a total time budget, retry in **one** layer (§6.7).

**More decisions in one line each**

| Question | Answer | § |
|---|---|---|
| `@Value` or `@ConfigurationProperties`? | one value / SpEL → `@Value`; a group of settings → a `@ConfigurationProperties` record | §3.6 |
| `@Component` or `@Bean`? | your class → stereotype; third-party class or creation logic → `@Bean` | §2.3 |
| Unit test, slice or `@SpringBootTest`? | logic → unit; Spring-produced behaviour of one layer → slice; wiring → a few `@SpringBootTest`s | §12.2 |
| `Page` or `Slice` (or `Window`)? | page numbers → `Page` (extra COUNT); "load more" → `Slice`; big tables → keyset `Window` | §8.11 |
| `@EventListener`, `@TransactionalEventListener` or outbox? | in-process now → `@EventListener`; only after commit → `@TransactionalEventListener`; must not be lost → outbox | §2.12, §7.15, §14.17 |
| JWT or opaque token? | local fast validation → JWT; instant revocation / hidden claims → opaque + introspection | §11.7 |
| Session cookie or tokens? | browser app / BFF → session (+ CSRF); service-to-service, mobile → bearer tokens | §10.10, §11.12 |
| Blocking or non-blocking retries (Kafka)? | order matters → `DefaultErrorHandler` (blocking); independent records → `@RetryableTopic` | §14.15 |
| Flyway or Liquibase? | SQL-first, one database → Flyway; rollbacks, preconditions, many databases → Liquibase | §9.4 |
| Eureka or Kubernetes Services? | on Kubernetes, Services + DNS usually suffice | §16.4 |
| OpenFeign or HTTP interfaces? | new code → HTTP interfaces (OpenFeign is in maintenance mode) | §16.6 |
| JVM + AOT cache or native image? | long-running, throughput-heavy → JVM (AOT cache halves startup); CLIs, scale-to-zero → native | §15.11 |

---

## 19.13 Boot 3 → 4 quick migration checklist

Work top to bottom; each step names the symptom you see if you skip it. Details: §17.12, §17.13, §17.14, §17.15.

**Before you start**

- [ ] Upgrade to **Boot 3.5.x** first and remove every deprecation warning (4.0 deletes what 3.x deprecated). From 2.7, go 2.7 → 3.5 (`javax` → `jakarta`) first.
- [ ] Java 17+ (25 recommended); Kotlin 2.2+, GraalVM 25+, Gradle 8.14+ if you use them.
- [ ] Run OpenRewrite's `org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0` recipe and review the diff.
- [ ] Add `spring-boot-properties-migrator` (runtime, temporarily) to report renamed properties; remove it afterwards.
- [ ] Stuck? `spring-boot-starter-classic` / `spring-boot-starter-test-classic` restore the old all-in-one layout while you fix imports.

**Dependencies**

- [ ] `spring-boot-starter-web` → `spring-boot-starter-webmvc`; `-aop` → `-aspectj`; `-oauth2-*` → `-security-oauth2-*`; `-web-services` → `-webservices`.
- [ ] Add `spring-boot-starter-restclient` (client code) and `spring-boot-starter-webclient` (`WebClient.Builder`, else "No qualifying bean of type WebClient$Builder").
- [ ] Add a **test starter** per technology (`-webmvc-test`, `-data-jpa-test`, ...) — else `@WebMvcTest` does not resolve.
- [ ] Add `spring-boot-starter-flyway` / `-liquibase` — else migrations silently stop running.
- [ ] Undertow → Tomcat or Jetty. Spring Retry → declare its version or move to `@Retryable` (F7).
- [ ] Separately versioned projects: Spring Cloud **2025.1.x** (2025.1.2+ for Boot 4.1), Spring AI **2.0**, springdoc **3.x**, Testcontainers **2.x** artifact names.

**Code**

- [ ] Jackson 3: `com.fasterxml.jackson.databind` → `tools.jackson.databind` (annotations unchanged); `ObjectMapper` bean → `JsonMapper`; `Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`.
- [ ] Auto-configuration class imports moved to per-module packages (check `exclude = ...`, `@ImportAutoConfiguration`, `@AutoConfiguration(after = ...)`).
- [ ] `@EntityScan` → `org.springframework.boot.persistence.autoconfigure`; `hibernate-jpamodelgen` → `hibernate-processor`.
- [ ] Null-safety: `org.springframework.lang.Nullable` → `org.jspecify.annotations.Nullable`; `javax.annotation`/`javax.inject` are gone.

- [ ] Security 7: rewrite `.and()` chains as one lambda per feature, `authorizeRequests` → `authorizeHttpRequests`, Ant/MVC matchers → `PathPatternRequestMatcher` (plain `requestMatchers("/x/**")`).
- [ ] `RestTemplate` → `RestClient` (start with `RestClient.create(restTemplate)`).

**Tests**

- [ ] `@MockBean` / `@SpyBean` → `@MockitoBean` / `@MockitoSpyBean` (removed in Boot 4; not allowed inside `@Configuration` classes).
- [ ] `@SpringBootTest` + `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureWebTestClient` / `@AutoConfigureTestRestTemplate` — no longer automatic.
- [ ] Slice annotation imports moved (e.g. `org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest`); JUnit 4 runners → JUnit 6 (Jupiter).

**Properties (most common renames)**

| Boot 3 | Boot 4 |
|---|---|
| `spring.jackson.parser.*` / `generator.*` | `spring.jackson.json.read.*` / `json.write.*` |
| `spring.data.mongodb.uri` / `.host` / ... | `spring.mongodb.uri` / `.host` / ... |
| `spring.http.client.*` | `spring.http.clients.*` |
| `server.error.*` (e.g. `include-stacktrace`, `path`) | `spring.web.error.*` |
| `spring.dao.exceptiontranslation.enabled` | `spring.persistence.exceptiontranslation.enabled` |
| `management.tracing.enabled` | `management.tracing.export.enabled` |
| `management.otlp.tracing.endpoint` | `management.opentelemetry.tracing.export.otlp.endpoint` |
| `management.health.mongo.enabled` | `management.health.mongodb.enabled` |
| `spring.session.redis.*` | `spring.session.data.redis.*` |
| `spring.kafka.retry.topic.backoff.random` | `spring.kafka.retry.topic.backoff.jitter` |
| `spring.rabbitmq.listener.simple.retry.max-attempts` (counted the first call) | `...retry.max-retries` (does not) |

**Behaviour changes to re-test**

- [ ] Health probes (`/actuator/health/liveness`, `/readiness`) are **on by default** — check what your platform calls.
- [ ] DevTools LiveReload is off (and deprecated in 4.1).
- [ ] `server.forward-headers-strategy` no longer applies to WAR deployments.
- [ ] Jackson 3 defaults: ISO dates, unknown properties ignored, properties sorted alphabetically after creator properties.

---

## 19.14 Interview index

Chapter 18 holds 168 questions with short answers, five runnable proofs, a project walkthrough and ten debugging scenarios. This index maps each topic to its Chapter 18 section and to the chapter that teaches it.

| Topic | Chapter 18 section | Questions | Teaching chapter |
|---|---|---|---|
| IoC, beans, scopes, lifecycle, events | §18.2 | Q1–Q20 | Chapter 2 |
| Boot, starters, configuration, auto-configuration | §18.3 | Q21–Q35 | Chapter 1, Chapter 3 |
| AOP, proxies, Framework 7 resilience | §18.4 | Q36–Q44 | Chapter 4 |
| MVC, REST, validation, errors, versioning, clients | §18.5 | Q45–Q61 | Chapter 5, Chapter 6 |
| JDBC, transactions, JPA, Spring Data | §18.6 | Q62–Q91 | Chapter 7, Chapter 8, Chapter 9 |
| Caching | §18.7 | Q92–Q97 | Chapter 9 |
| Security, JWT, OAuth2 | §18.8 | Q98–Q114 | Chapter 10, Chapter 11 |
| Testing | §18.9 | Q115–Q125 | Chapter 12 |
| Reactive and WebFlux | §18.10 | Q126–Q132 | Chapter 13 |
| Messaging, async, scheduling | §18.11 | Q133–Q140 | Chapter 14 |
| Production and observability | §18.12 | Q141–Q152 | Chapter 15 |
| Microservices and Spring Cloud | §18.13 | Q153–Q160 | Chapter 16 |
| Spring AI and Boot 4 / Framework 7 changes | §18.14 | Q161–Q168 | Chapter 17 |
| Proofs of five tricky answers | §18.15 | prototype-in-singleton, `@Qualifier` vs `@Primary`, cycles, checked exceptions, self-invocation | this chapter's §19.15 adds more |
| "Explain your Spring project" template | §18.16 | — | — |
| Ten debugging scenarios | §18.17 | bean not found, cycles, no rollback, lazy loading, N+1, 401/403, CORS, "annotation does nothing", auto-configuration failures, slow startup | — |

**The twenty questions that come up most**

| # | Question | Short answer | Study |
|---|---|---|---|
| 1 | IoC vs DI? | framework creates objects / objects receive dependencies | §2.1 |
| 2 | `BeanFactory` vs `ApplicationContext`? | lazy minimal container / eager, events, environment, post-processors | §2.2 |
| 3 | Why constructor injection? | `final`, complete objects, testable with `new`, cycles fail fast | §2.4 |
| 4 | Two beans of one type? | `@Qualifier` > `@Primary` > `@Fallback` > parameter name | §2.5 |
| 5 | Singleton thread-safe? prototype in singleton? | shared → keep stateless / created once, use `ObjectProvider` | §2.6 |
| 6 | Bean lifecycle? | §19.7 order | §2.7 |
| 7 | How does auto-configuration work? | `.imports` files, conditions, applied after your beans, `@ConditionalOnMissingBean` | §3.10 |
| 8 | Property precedence? | command line > system props > env vars > profile files > `application.*` | §3.3 |
| 9 | JDK vs CGLIB proxy; self-invocation? | interfaces vs subclass (Boot default CGLIB) / `this.x()` skips the proxy | §4.4, §4.5 |
| 10 | Request flow in MVC? | filters → `DispatcherServlet` → mapping → interceptors → arguments → controller → converter → resolvers | §5.2 |
| 11 | Global error handling? | `@RestControllerAdvice` + `ProblemDetail` (RFC 9457) | §5.7 |
| 12 | How does `@Transactional` work? | proxy + manager + thread-bound connection | §7.7 |
| 13 | Propagation types; checked exception? | §19.6 / commits by default | §7.9, §7.10 |
| 14 | N+1 and `LazyInitializationException`? | JOIN FETCH / `@EntityGraph` / batch size; fetch per use case, DTOs | §8.7, §8.6 |
| 15 | Security filter chain; 401 vs 403? | `DelegatingFilterProxy` → `FilterChainProxy` → first matching chain | §10.2, §10.6 |
| 16 | JWT validation in a resource server? | signature via JWKS, `exp` / `nbf` (60 s skew), `iss`, `aud`, `scope` → `SCOPE_` | §11.3 |
| 17 | Slices vs `@SpringBootTest`; context caching? | partial vs full context / cache keyed by configuration, max 32 | §12.7, §12.11 |
| 18 | WebFlux or virtual threads? | MVC + virtual threads for CRUD; WebFlux for streaming | §13.15 |
| 19 | Reliable DB write + event? | transactional outbox + idempotent consumers | §14.17 |
| 20 | What's new in Boot 4 / Framework 7? | modular starters, Jackson 3, JSpecify, API versioning, HTTP service clients, `@Retryable`, `RestTestClient` | §17.13, §17.14 |

---

## 19.15 Proofs: the facts on these sheets, run

### What it is

`code/cheatsheet-checks/` is a small Maven project (Boot 4.1.1, Java 25, no web server, no Docker) whose tests assert the facts marked ✔ in this chapter. If a future Boot version changes a default, a test fails, and the sheet must be updated. Run it from the repository root with `bash tools_run_spring.sh spring '19-*'`.

| Test class | Proves | Sheet |
|---|---|---|
| `ScopeAndLifecycleTests` | singleton shared, prototype fresh, prototype-in-singleton stuck, init/destroy order, no destroy callback for prototypes | §19.7 |
| `BeanSelectionTests` | `@Qualifier` beats `@Primary`, `@Primary` beats the parameter name, `List` by `@Order`, `Map` by bean name | §19.3, §19.7 |
| `TransactionTests` | `@Transactional` defaults, all seven propagation types, rollback rules, `UnexpectedRollbackException` | §19.6 |
| `PropertyBindingTests` | env-var mapping (canonical and legacy), list index, `Duration` / `DataSize` units, record + `@DefaultValue` | §19.5 |
| `PropertyPrecedenceTests` | defaults < `application.properties` < env var < `-D` < `--arg`; `@Value` defaults and SpEL | §19.5 |
| `BootDefaultsTests` | HikariCP pool 10 / timeout 30 s, unique embedded DB name, `@Async` executor 8 threads `task-` (virtual threads → `SimpleAsyncTaskExecutor`), scheduler 1 thread, shutdown 30 s | §19.5 |
| `CoreUtilitiesTests` | default cache keys, six-field cron, `@Retryable` defaults and 1 + `maxRetries` attempts | §19.3, §19.12 |
| `JsonAndSecurityDefaultsTests` | Jackson 3 defaults, `hasRole` prefix rules, `DelegatingPasswordEncoder` | §19.5, §19.8 |

The transaction checks use plain Spring on an in-memory H2 database, the same technique as §18.15:

```java
@Transactional
void catchInnerRequired() {
    inner.insert("outer");
    try { inner.requiredThenFail(); } catch (IllegalStateException swallowed) { }
}   // commit attempt -> UnexpectedRollbackException, nothing is saved
```

The environment-variable checks bind through the same `Binder` that `@ConfigurationProperties` uses, over a fake `SystemEnvironmentPropertySource`:

```java
Binder canonical = envBinder(Map.of("SHOP_CHECKOUTTIMEOUT", "2m", "SHOP_PAYMENT_RETRIES", "4", "SHOP_CURRENCIES_0", "USD"));
Duration timeout = canonical.bind("shop.checkout-timeout", Duration.class).get();   // PT2M
```

**Output**

`mvn -q -B -ntp verify` (20 tests, all passing; JUnit does not fix the order of test classes, so blocks may appear in another order):

```
spring.task.scheduling.pool.size default = 1, spring.lifecycle.timeout-per-shutdown-phase default = PT30S
applicationTaskExecutor = ThreadPoolTaskExecutor, core = 8, prefix = task-, queue = unbounded
with spring.threads.virtual.enabled=true -> SimpleAsyncTaskExecutor
DataSource = HikariDataSource, maximumPoolSize = 10, connectionTimeout = 30000 ms
no spring.datasource.url + H2 on the classpath -> jdbc:h2:mem:<random name>  (generate-unique-name=true)
'0 30 2 * * MON-FRI' after 2026-09-28T10:00 -> 2026-09-29T02:30
five-field Unix cron '0 2 * * *' -> IllegalArgumentException
SimpleKeyGenerator: () -> SimpleKey [], ("BOOK-1") -> BOOK-1, ("BOOK-1","EUR") -> SimpleKey [BOOK-1, EUR]
@Retryable defaults: maxRetries=3, delay=1000 ms, multiplier=1.0
maxRetries = 2 -> [attempt 1, attempt 2, attempt 3], then the last exception is rethrown
SHOP_CHECKOUTTIMEOUT=2m   -> shop.checkout-timeout = PT2M
SHOP_PAYMENT_RETRIES=4    -> shop.payment.retries  = 4
SHOP_CURRENCIES_0=USD     -> shop.currencies[0]    = USD
SHOP_CHECKOUT_TIMEOUT=3m  -> shop.checkout-timeout = PT3M (legacy '-' -> '_' form, also accepted)
Duration '45s' -> PT45S, '500' -> 500 ms, 'PT1H' -> PT1H
DataSize '10MB' -> 10485760 bytes, '1024' -> 1024 bytes
record bound with @DefaultValue: ShopProperties[name=Corner Shop, checkoutTimeout=PT30S, maxUpload=5242880B]
init order    : [constructor, @PostConstruct, afterPropertiesSet, initMethod]
destroy order : [@PreDestroy, destroy(), destroyMethod]  (no Proto @PreDestroy)
singleton: getBean twice -> same instance?        true
prototype: getBean twice -> new instance?         true
prototype injected into a singleton -> same one?  true
@Value ${shop.missing:fallback} -> fallback, #{${shop.retries:3} + 1} with retries=2 -> 3, key set only in defaultProperties -> eu
(default)
defaultProperties + application.properties -> from application.properties
+ env SHOP_NAME                           -> from env var
+ -Dshop.name                             -> from -D system property
+ --shop.name                             -> from --command-line
by type (@Primary)        -> sms
@Qualifier("email")        -> email
parameter named 'push'    -> sms  (@Primary wins over the name)
List<Notifier> by @Order  -> [email, sms, push]
Map keys (bean names)     -> [email, push, sms]
encode -> {bcrypt}$2a$10$...  matches {noop}s3cret? true  upgradeEncoding({noop}) = true
hasRole("ADMIN"): authority ROLE_ADMIN -> true, authority ADMIN -> false
hasRole("ROLE_ADMIN") -> IllegalArgumentException
JsonMapper (tools.jackson.databind.json) writes {"item":"book","on":"2026-09-28"}
unknown property ignored -> Delivery[item=pen, on=2026-10-01]
broken JSON -> JacksonException (unchecked)
committed despite an exception: [checked, default rules, caught inside the method]
inner REQUIRED failed, outer caught it -> UnexpectedRollbackException, rows = []
@Transactional defaults: propagation=REQUIRED isolation=DEFAULT readOnly=false timeout=-1
REQUIRED      : inner runs in the outer transaction? true (Outer.requiredJoins)
REQUIRES_NEW  : outer rolled back, rows left = [audit (REQUIRES_NEW)]
NESTED        : inner failed + caught, rows left = [outer] (savepoint)
MANDATORY     : no transaction -> IllegalTransactionStateException
NEVER         : inside a transaction -> IllegalTransactionStateException
SUPPORTS      : active alone? false, active inside a transaction? true
NOT_SUPPORTED : active inside a transaction? false (outer suspended)
```

**What the output teaches**

- A **legacy** environment-variable spelling (`SHOP_CHECKOUT_TIMEOUT`, dash turned into an underscore) also binds to `shop.checkout-timeout`. The documented form without the dash separator (`SHOP_CHECKOUTTIMEOUT`) is the one to use and to search for; know that both work when you hunt for "who set this value?".
- `@Primary` beats a matching parameter name — the name is only the **last** tie-breaker.
- HikariCP reports its pool size only after the pool has started; the test takes one connection first.
- The `@Retryable` bean is a CGLIB proxy, so the test reads its attempt list through a method: a field read on a proxy returns the proxy's own, never-initialised field (§4.5).

---

## 19.16 Glossary A–Z

Short definitions of the terms used in this book. Annotations are listed without `@` in the sort order. Follow the link for the full explanation.

**A**

| Term | Meaning | See |
|---|---|---|
| Access (Actuator) | per-endpoint permission: `none`, `read-only`, `unrestricted` (Boot 3.4+); separate from exposure | §15.2 |
| `acks` (Kafka) | how many replicas must confirm a write; `all` (`-1`) is the default since Kafka 3.0 | §14.16 |
| Actuator | Boot's management endpoints: health, metrics, info, loggers, env... | §15.2 |
| Advice (AOP) | the code an aspect runs at a join point: before, after returning, after throwing, after, around | §4.1 |
| Advisor (AOP) | Spring's internal pair of one advice and one pointcut | §4.1 |
| Advisor (Spring AI) | interceptor around every `ChatClient` call (memory, RAG, tool calling, logging), ordered by `getOrder()` | §17.5 |
| Aggregate | DDD cluster of objects loaded and saved as one unit through its root; the unit of Spring Data JDBC | §9.2 |
| `AggregateReference` | Spring Data JDBC reference to another aggregate by id only | §9.2 |
| AOP | Aspect-Oriented Programming: write a cross-cutting concern once, apply it where a pointcut matches | §4.1 |
| AOT cache (JDK) | JDK 24+ file of classes loaded and linked in a training run; `-XX:AOTCacheOutput` / `-XX:AOTCache` | §15.11 |
| AOT processing (Spring) | build-time generation of bean-definition code, used by native images and optionally on the JVM | §15.11 |
| API gateway | single front door that routes requests and applies edge concerns (Spring Cloud Gateway) | §16.7 |
| API versioning | Framework 7 feature: read a version from the request and match `version = "1.1+"` mappings | §5.9 |
| `ApiVersionInserter` | client-side strategy that puts the API version into requests | §6.7 |
| `ApplicationContext` | Spring's full container: eager singletons, post-processors, events, `Environment`, i18n | §2.2 |
| `ApplicationContextRunner` | test helper that builds a tiny context per test, ideal for auto-configuration | §3.13 |
| `ApplicationEventPublisher` | bean used to publish application events | §2.12 |
| `ApplicationReadyEvent` | Boot event after startup and runners have finished | §2.7 |
| `ApplicationRunner` / `CommandLineRunner` | beans run once at startup with the program arguments | §1.8 |
| Aspect | class bundling pointcuts and advice for one concern (`@Aspect` + `@Component`) | §4.2 |
| AspectJ | full AOP language with compile/load-time weaving; Spring borrows its annotations and pointcut syntax | §4.8 |
| At-least-once | delivery guarantee: never lost, possibly duplicated — consumers must be idempotent | §14.8 |
| `Authentication` | object holding the principal, credentials and authorities of the current caller | §10.1 |
| Authentication | proving who the caller is | §10.1 |
| `AuthenticationEntryPoint` | starts authentication for an anonymous, denied request: 401 challenge or login redirect | §10.2 |
| `AuthenticationManager` | entry point of authentication; usually a `ProviderManager` asking `AuthenticationProvider`s | §10.4 |
| Authority (`GrantedAuthority`) | one permission string held by the caller (`report:read`, `ROLE_ADMIN`, `SCOPE_x`) | §10.1 |
| Authorization | deciding whether a known caller may perform an action | §10.1 |
| Authorization code flow | OAuth2 flow for apps with a user: code in the browser, token exchange on the back channel (with PKCE) | §11.9 |
| Authorization server | issues tokens (Keycloak, Okta, Entra ID, Spring Authorization Server) | §11.2 |
| `AuthorizationFilter` | last filter of a security chain; applies `authorizeHttpRequests` rules | §10.2 |
| Auto-configuration | conditional `@Configuration` classes that Boot applies after yours to fill gaps | §3.10 |
| `AutoConfiguration.imports` | file `META-INF/spring/...AutoConfiguration.imports` listing a jar's auto-configurations | §3.10 |
| Availability state | Boot's `LivenessState` and `ReadinessState`, exposed as health groups | §15.3 |
| Awaitility | test library that polls a condition until true or timeout | §14.2 |
| `Aware` callbacks | interfaces (`BeanNameAware`, `ApplicationContextAware`) through which a bean receives container objects | §2.7 |

## B

| Term | Meaning | See |
|---|---|---|
| Backpressure | the consumer limits the producer via `request(n)` | §13.7 |
| Baggage | extra key/value context that travels with a trace | §15.7 |
| Bean | object created, wired and managed by the Spring container | §2.1 |
| Bean definition | the recipe for a bean: class or factory method, scope, dependencies, callbacks | §2.2 |
| `BeanFactory` | minimal container: bean definitions + lazy creation | §2.2 |
| `BeanFactoryPostProcessor` | changes bean **definitions** before any bean is created (e.g. `${}` resolution) | §2.8 |
| `BeanPostProcessor` | sees every bean **instance** before/after init and may wrap it in a proxy | §2.8 |
| `BeanRegistrar` | Framework 7 contract for registering beans programmatically (loops, conditions) | §2.14 |
| Bearer token | token that grants access to whoever holds it, sent as `Authorization: Bearer ...` | §11.1 |
| BFF (Backend for Frontend) | server-side OAuth2 client that keeps tokens away from the browser and gives it a session cookie | §11.12 |
| BlockHound | Java agent that detects blocking calls on non-blocking threads in tests | §13.8 |
| BOM | "bill of materials" POM pinning versions of many artifacts (`spring-boot-dependencies`) | §1.6 |
| `boundedElastic` | Reactor scheduler for wrapping unavoidable blocking calls | §13.8 |
| Bulkhead | cap on concurrent calls to one dependency (`@ConcurrencyLimit`, Resilience4j `@Bulkhead`) | §4.7, §16.8 |
| Buildpacks | Cloud Native Buildpacks: build an OCI image from a jar without a Dockerfile (`spring-boot:build-image`) | §15.11 |

## C

| Term | Meaning | See |
|---|---|---|
| Cache-aside | look in the cache; on a miss load, store and return — what `@Cacheable` does | §9.8 |
| Cache stampede | many concurrent misses for one hot key all compute it; `sync = true` mitigates per JVM | §9.8 |
| `CacheManager` | provider-neutral cache interface behind the cache annotations | §9.8 |
| Caffeine | high-performance local Java cache (W-TinyLFU eviction) | §9.8 |
| Cardinality | number of distinct tag-value combinations of a metric; keep it small | §15.5 |
| Cascade | passing a JPA operation (persist, merge, remove...) from parent to children | §8.5 |
| CDS | Class Data Sharing: JVM archive of parsed classes for faster startup | §15.11 |
| CGLIB proxy | runtime subclass used as a proxy; Boot's default; cannot handle `final` | §4.4 |
| `ChatClient` | Spring AI fluent API over a `ChatModel`, with advisors, templates, tools | §17.3 |
| `ChatMemory` | Spring AI store of past messages per conversation id (window of 20 by default) | §17.6 |
| `ChatModel` | Spring AI provider-level interface: `Prompt` in, `ChatResponse` out | §17.3 |
| Circuit breaker | stops calling a failing dependency (CLOSED → OPEN → HALF_OPEN) | §16.8 |
| Circular dependency | dependency cycle between beans; constructor cycles always fail, Boot rejects field cycles | §2.9 |
| Claim | statement in a JWT payload (`sub`, `iss`, `aud`, `exp`, `scope`) | §11.1 |
| Client credentials | OAuth2 grant for service-to-service calls without a user | §11.10 |
| Cold publisher | replays the whole pipeline for every subscriber | §13.9 |
| Component scanning | finding `@Component` classes in a package tree | §2.3 |
| `ConcurrencyLimit` | Framework 7 annotation limiting concurrent calls of a method (`BLOCK` or `REJECT`) | §4.7 |
| Condition evaluation report | Boot's list of which auto-configurations matched and why (`--debug`, `/actuator/conditions`) | §3.11 |
| `Conditional` | register a bean only if a `Condition` matches; base of all `@ConditionalOn...` | §2.11 |
| Config data | Boot's files and imports: `application.*`, profile files, `spring.config.import` | §3.3 |
| Config Server | Spring Cloud service serving configuration from Git or other backends | §16.3 |
| `configtree:` | import where each file in a directory is a property (Kubernetes secrets) | §3.8 |
| `ConfigurationProperties` | binds a prefix of properties into a typed, validated object | §3.6 |
| Consumer group | Kafka consumers sharing a `group.id`; each partition goes to one member, each group gets every record | §14.9 |
| Content negotiation | choosing the representation from `Accept` / `produces` (406 if impossible) | §5.8 |
| Context caching (tests) | the TestContext Framework reuses started contexts with the same configuration | §12.11 |
| Context pausing | Framework 7: inactive cached test contexts have their lifecycle beans stopped | §12.11 |
| Context propagation | copying `ThreadLocal` values (trace, MDC) across threads or into the Reactor `Context` | §15.6, §13.9 |
| Contract testing | consumer and provider tested against a shared contract (Pact, Spring Cloud Contract) | §16.10 |
| CORS | browser rules letting a page from one origin read responses from another (preflight `OPTIONS`) | §10.9 |
| Cron expression | Spring's six-field schedule, starting with seconds (`0 0 2 * * *`) | §14.7 |
| Cross-cutting concern | requirement spread over many classes (logging, transactions, security) | §4.1 |
| CSRF | attack that makes a logged-in browser send a forged state-changing request; blocked by tokens | §10.8 |
| Customizer | bean that adjusts an auto-configured object instead of replacing it | §3.12 |

**D**

| Term | Meaning | See |
|---|---|---|
| `DataAccessException` | Spring's unchecked, database-neutral exception hierarchy | §7.5 |
| `DataSize` | Boot type for sizes (`10MB`); plain numbers are bytes | §3.6 |
| `DataSource` | factory of JDBC connections; in Boot a HikariCP pool | §7.2 |
| Dead-letter topic (DLT) | where records go after the last failed attempt; default name `<topic>-dlt` (spring-kafka 3.3+) | §14.14 |
| `DeadLetterPublishingRecoverer` | spring-kafka recoverer that publishes failed records to the DLT | §14.14 |
| `DefaultErrorHandler` | spring-kafka container error handler: blocking retries, then a recoverer | §14.14 |
| `DelegatingFilterProxy` | servlet filter that hands requests to the `springSecurityFilterChain` bean | §10.2 |
| `DelegatingPasswordEncoder` | encoder storing `{id}hash`; verifies old formats, upgrades on login | §10.5 |
| Dependency Injection (DI) | an object receives its dependencies instead of creating them | §2.1 |
| Derived query | Spring Data method whose name is the query (`findByTitleContaining`) | §8.9 |
| Detached entity | JPA entity no longer in a persistence context; changes are not tracked | §8.4 |
| DevTools | development module: fast restart, dev defaults; off for `java -jar` | §1.10 |
| Dirty checking | Hibernate compares managed entities with their snapshots at flush and writes `UPDATE`s | §8.4 |
| `DirtiesContext` | test annotation that closes and evicts the cached context | §12.11 |
| `DiscoveryClient` | Spring Cloud abstraction returning the instances of a service id | §16.4 |
| `DispatcherServlet` | Spring MVC's front controller | §5.2 |
| `DispatcherHandler` | WebFlux's front controller | §13.10 |
| Dual-write problem | a database write and a broker publish cannot be atomic; solved by the outbox | §14.17 |
| `Duration` binding | `45s`, `5m`, `PT1H`; plain numbers are milliseconds | §3.6 |
| `DynamicPropertySource` | test method registering properties with lazy suppliers (random ports, containers) | §12.12 |

**E**

| Term | Meaning | See |
|---|---|---|
| EAGER / LAZY | load an association with its owner / on first access; make everything LAZY | §8.6 |
| Embedded server | Tomcat/Jetty/Netty running inside the application jar | §1.2 |
| `EmbeddedKafka` | in-JVM Kafka broker for tests (spring-kafka-test) | §14.13 |
| Embedding | vector of numbers representing text so similar texts are close | §17.9 |
| `EnableAutoConfiguration` | imports Boot's auto-configuration candidates | §3.10 |
| `EnableResilientMethods` | Framework 7 switch for `@Retryable` and `@ConcurrencyLimit` | §4.7 |
| Entity | JPA class whose instances are table rows (`@Entity`, `@Id`, no-arg constructor, not `final`) | §8.3 |
| Entity graph | declares associations to fetch with a query (`@EntityGraph`, left join) | §8.7 |
| `EntityManager` | JPA API over the persistence context (Hibernate `Session`) | §8.4 |
| `Environment` | Spring's view of all property sources and active profiles | §3.1 |
| Eureka | Netflix service registry supported by Spring Cloud (AP design) | §16.4 |
| Event loop | few threads that run pipeline steps when I/O completes, never waiting | §13.1 |
| `EventListener` | method receiving application events, synchronously by default | §2.12 |
| Exception translation | turning `SQLException`s into `DataAccessException`s | §7.5 |
| `ExceptionTranslationFilter` | turns security exceptions into 401/redirect (anonymous) or 403 (authenticated) | §10.2 |
| Executor / `TaskExecutor` | owner of worker threads and a task queue; runs `@Async` work | §14.4 |
| Exposure (Actuator) | whether an endpoint is reachable over HTTP or JMX (`health` only, by default) | §15.2 |
| Externalized configuration | settings outside the build artifact (files, env vars, args) | §3.1 |

## F

| Term | Meaning | See |
|---|---|---|
| `FACTOR_*` authority | Security 7 authority recording a completed login factor (`FACTOR_PASSWORD`) | §10.4 |
| `Fallback` | Framework 6.2: bean used only if no other candidate qualifies | §2.5 |
| Fat (executable) jar | Boot jar with `BOOT-INF/lib` dependencies, started by `JarLauncher` | §1.9 |
| Filter (servlet) | code around every request before the `DispatcherServlet`; can wrap request/response | §5.11 |
| `FilterChainProxy` | the `springSecurityFilterChain` bean that picks the first matching `SecurityFilterChain` | §10.2 |
| `fixedDelay` / `fixedRate` | `@Scheduled` timing: end-to-start / start-to-start | §14.7 |
| `flatMap` / `concatMap` | async per item: concurrent and interleaved / sequential and ordered | §13.4 |
| Flush | Hibernate sends queued SQL: before commit, before affected queries, on `flush()` | §8.4 |
| `Flux` | Reactor publisher of 0..N items | §13.3 |
| Flyway | SQL-file schema migration tool (`V1__x.sql`, checksums) | §8.16 |
| Form login | username/password via an HTML form; session cookie afterwards | §10.4 |
| Full vs lite configuration | `@Configuration` CGLIB subclass (inter-bean calls return singletons) vs plain calls | §2.3 |
| Functional endpoints | routes as `RouterFunction` beans and handler functions instead of annotations | §13.11 |

## G – H

| Term | Meaning | See |
|---|---|---|
| Gateway (Spring Cloud) | API gateway: routes = predicates + filters; Web MVC or WebFlux flavour | §16.7 |
| Gauge | Micrometer meter sampling a current value | §15.5 |
| Graceful shutdown | stop accepting requests, finish in-flight ones, then close (default since Boot 3.4) | §15.9 |
| GraalVM native image | whole app compiled ahead of time to a native executable (closed world) | §15.11 |
| `HandlerInterceptor` | MVC hook around handler methods (`preHandle`, `postHandle`, `afterCompletion`) | §5.11 |
| `HandlerMapping` | finds the handler method for a request | §5.2 |
| Health group | named subset of health indicators at `/actuator/health/<group>` | §15.3 |
| `HealthIndicator` | bean contributing one check to `/actuator/health` | §15.3 |
| HikariCP | Boot's default JDBC connection pool (default 10 connections) | §7.2 |
| Hot publisher | produces whether or not anyone listens; late subscribers miss items | §13.9 |
| HTTP Basic | credentials in an `Authorization: Basic` header on every request | §10.4 |
| HTTP service interface | Java interface with `@GetExchange`... implemented by a generated proxy | §6.6 |
| `HttpMessageConverter` | converts bodies to and from Java objects (Jackson for JSON) | §5.2 |

## I – J

| Term | Meaning | See |
|---|---|---|
| Idempotent | repeating the operation gives the same result (GET, PUT, DELETE; not POST) | §5.1 |
| Idempotent consumer | de-duplicates messages by id so at-least-once delivery is safe | §14.17 |
| Idempotent producer | Kafka producer whose retries cannot duplicate records (on by default) | §14.16 |
| Identity provider (IdP) | system that owns user accounts and issues tokens | §11.2 |
| `IDENTITY` / `SEQUENCE` | JPA id strategies: auto-increment (no insert batching) / database sequence | §8.3 |
| `ImportHttpServices` | Framework 7 registration of HTTP interface groups; Boot configures them by properties | §6.6 |
| Introspection | asking the authorization server whether an opaque token is active (RFC 7662) | §11.7 |
| Inversion of Control (IoC) | the framework, not your code, creates objects and calls you | §2.1 |
| Isolation level | how much a transaction sees of concurrent ones (READ COMMITTED...) | §7.11 |
| Jackson 3 | Boot 4's JSON library, packages `tools.jackson.*` | §5.5 |
| Jakarta EE | successor of Java EE; `jakarta.*` packages (EE 11 in Boot 4) | §1.4 |
| `JarLauncher` | `Main-Class` of a Boot fat jar that loads nested jars and calls your `Start-Class` | §1.9 |
| `JdbcClient` | Framework 6.1 fluent API for SQL with automatic record mapping | §7.3 |
| `JdbcTemplate` | classic SQL template; still used for batch updates | §7.3 |
| JDK dynamic proxy | `java.lang.reflect.Proxy` implementing the bean's interfaces only | §4.4 |
| Jitter | random variation added to retry delays so clients don't retry in lockstep | §4.7 |
| Join point | a point where advice can run; in Spring AOP always a bean method execution | §4.1 |
| `JOIN FETCH` | JPQL that loads an association in the same query (N+1 fix) | §8.7 |
| JPA | Jakarta Persistence specification; Hibernate implements it | §8.1 |
| JPQL | query language over entities and fields, not tables | §8.10 |
| `JsonMapper` | Jackson 3's immutable JSON mapper, built with `JsonMapper.builder()` | §5.5 |
| JSpecify | standard null-safety annotations (`@NullMarked`, `@Nullable`) used by Framework 7 | §2.13 |
| JWK Set (JWKS) | JSON document of an issuer's public keys, selected by `kid` | §11.1 |
| JWT | signed token `header.payload.signature`; claims readable by anyone | §11.1 |
| `JwtAuthenticationConverter` | turns JWT claims into authorities (`scope` → `SCOPE_x`) | §11.5 |
| `JwtDecoder` | parses and validates a JWT (signature, `exp`, `iss`, custom validators) | §11.3 |

## K – L

| Term | Meaning | See |
|---|---|---|
| Kafka | distributed, replicated commit log for event streaming | §14.9 |
| `KafkaListener` | spring-kafka consumer method | §14.12 |
| `KafkaTemplate` | spring-kafka producer API; `send` returns a `CompletableFuture` | §14.11 |
| Key (Kafka) | decides the partition; same key → same partition → ordered | §14.9 |
| KRaft | Kafka's built-in consensus replacing ZooKeeper (Kafka 4.0+) | §14.9 |
| Lazy initialization | create beans on first use (`@Lazy`, `spring.main.lazy-initialization`) | §2.10 |
| `LazyInitializationException` | touching a lazy association after its persistence context closed | §8.6 |
| Lifecycle callbacks | init/destroy hooks: `@PostConstruct`, `afterPropertiesSet`, `@PreDestroy`... | §2.7 |
| Liquibase | changelog-based migration tool with rollbacks and preconditions | §9.4 |
| Liveness probe | "restart me?" — must depend only on internal state | §15.3 |
| Load balancing (client-side) | the caller picks an instance per request (Spring Cloud LoadBalancer, round robin) | §16.5 |
| `Lookup` | method injection returning a fresh prototype per call | §2.6 |
| Lost update | two read-modify-write cycles overwrite each other; prevent with SQL math, locks or `@Version` | §7.11 |

**M**

| Term | Meaning | See |
|---|---|---|
| Managed entity | entity tracked by the persistence context; changes are flushed automatically | §8.4 |
| `mappedBy` | marks the inverse side of a JPA relationship; only the owning side is written | §8.5 |
| `MappedCollection` | Spring Data JDBC child collection mapping (`idColumn`, `keyColumn`) | §9.2 |
| MCP | Model Context Protocol: standard way to expose and call AI tools across processes | §17.10 |
| MDC | Mapped Diagnostic Context: per-thread log fields (request id, trace id) | §14.4, §15.1 |
| `merge` vs `persist` | copy a detached entity's state onto a managed copy / make a new entity managed | §8.4 |
| `MeterRegistry` | Micrometer registry where meters are recorded and exported | §15.5 |
| Method security | authorization on bean methods (`@PreAuthorize`...), proxy-based | §10.7 |
| Method validation | constraints on method parameters; built into MVC since Framework 6.1 | §5.6 |
| Micrometer | vendor-neutral metrics facade ("SLF4J for metrics") | §15.1 |
| Micrometer Tracing | tracing facade over OpenTelemetry or Brave | §15.7 |
| MFA | multi-factor authentication; Security 7 tracks factors as `FACTOR_*` authorities | §10.4 |
| `MockitoBean` / `MockitoSpyBean` | mock / spy registered in the test context (replaced `@MockBean` / `@SpyBean`) | §12.10 |
| `MockMvc` / `MockMvcTester` | in-process MVC test client; `MockMvcTester` adds AssertJ assertions (F6.2) | §12.8 |
| `MockRestServiceServer` | fake server bound to a `RestClient.Builder` in `@RestClientTest` | §6.8 |
| Modular starters | Boot 4: one starter, one auto-configuration module and one test starter per technology | §3.14 |
| `Mono` | Reactor publisher of 0..1 item | §13.3 |
| Multi-tenancy (JWT) | several issuers in one API; `JwtIssuerAuthenticationManagerResolver` with an allow-list | §11.8 |

**N – O**

| Term | Meaning | See |
|---|---|---|
| N+1 problem | 1 query for parents + 1 per parent for a lazy association | §8.7 |
| `NESTED` | propagation using a savepoint inside the current transaction | §7.9 |
| `NimbusJwtEncoder` | Spring Security's JWT signer; Security 7 adds `withKeyPair`/`withSecretKey` builders | §11.4 |
| Nonce | OIDC random value bound into the ID token against replay | §11.9 |
| `NullMarked` / `Nullable` | JSpecify: non-null by default in a scope / this may be null | §2.13 |
| OAuth 2.0 | framework for delegated authorization with access tokens | §11.2 |
| `ObjectProvider` | lazy/optional lookup of a bean (`getIfAvailable`, `getObject` for prototypes) | §2.5 |
| Observation API | Micrometer API: instrument once, get a metric and a span | §15.6 |
| `Observed` | annotation creating an observation around a method | §15.6 |
| Offset (Kafka) | a record's position in its partition; committed per consumer group | §14.9 |
| OIDC | OpenID Connect: identity layer on OAuth2 adding the ID token | §11.2 |
| Opaque token | random token string checked by introspection; instantly revocable | §11.7 |
| Open Session in View | persistence context kept open for the whole web request (`spring.jpa.open-in-view`) | §8.15 |
| OpenAPI | machine-readable API description; generated by springdoc | §5.12 |
| OpenFeign | Spring Cloud declarative client, in maintenance mode | §16.6 |
| OpenRewrite | automated refactoring recipes used for Boot upgrades | §17.12 |
| OpenTelemetry | open standard and SDK for traces/metrics; OTLP is its protocol | §15.7 |
| Optimistic locking | `@Version` column checked in `UPDATE ... WHERE version = ?` | §8.14 |
| `orphanRemoval` | delete a child removed from its parent's collection | §8.5 |
| Outbox (transactional) | write the event into an outbox table in the same transaction; a relay publishes it | §14.17 |
| Owning side | the relationship side holding the foreign key; the only one JPA writes | §8.5 |

**P**

| Term | Meaning | See |
|---|---|---|
| `Page` / `Slice` / `Window` | paged result with total (extra COUNT) / "has next" only / keyset scrolling | §8.11 |
| Partition | ordered, append-only log inside a Kafka topic; unit of parallelism | §14.9 |
| `PathPattern` | Spring's URL pattern matcher (`{id}`, `**`, `{*path}`); no trailing-slash match | §5.3 |
| `PathPatternRequestMatcher` | Security 7 request matcher using `PathPattern` syntax | §10.3 |
| Persistence context | the `EntityManager`'s first-level cache and unit of work | §8.4 |
| Pessimistic locking | `SELECT ... FOR UPDATE` via `@Lock(PESSIMISTIC_WRITE)` | §8.14 |
| PKCE | proof key that binds an authorization code to the client that started the flow | §11.9 |
| `PlatformTransactionManager` | Spring interface that begins, commits and rolls back transactions | §7.7 |
| Pointcut | predicate selecting join points (`execution(...)`, `within(...)`, `@annotation(...)`) | §4.3 |
| Poison pill | record that can never be deserialized; `ErrorHandlingDeserializer` sends it to the DLT | §14.10 |
| Precedence | which property source wins (command line > ... > defaults) | §3.3 |
| Preflight | the browser's `OPTIONS` request asking CORS permission | §10.9 |
| `Primary` | default bean when several candidates match | §2.5 |
| Principal | the "who" of a request: user, service or device | §10.1 |
| `ProblemDetail` | RFC 9457 error body: `type`, `title`, `status`, `detail`, `instance` | §5.7 |
| Profile | named environment selecting beans (`@Profile`) and files (`application-dev.yaml`) | §3.7 |
| Profile group | one profile name that activates several | §3.7 |
| Projection | query result with only the needed fields (interface or record) | §8.12 |
| Prompt template | text with `{placeholders}` filled at run time (Spring AI `PromptTemplate`) | §17.4 |
| Propagation | what a transactional method does when a transaction already exists | §7.9 |
| Property source | one named set of key/values inside the `Environment` | §3.1 |
| Prototype scope | new bean instance per lookup; no destroy callbacks | §2.6 |
| Proxy | stand-in object that runs extra behaviour and then calls the real bean | §4.4 |
| `Proxyable` | Framework 7 per-bean choice of JDK or CGLIB proxy | §4.4 |
| `publishOn` / `subscribeOn` | move downstream operators / the subscription to a scheduler | §13.8 |

## Q – R

| Term | Meaning | See |
|---|---|---|
| `Qualifier` | selects a specific bean at an injection point | §2.5 |
| `QuestionAnswerAdvisor` | Spring AI advisor doing simple RAG over a `VectorStore` | §17.9 |
| R2DBC | reactive (non-blocking) SQL API; not an ORM | §13.14 |
| RabbitMQ | queue broker: exchanges route messages to queues via bindings | §14.18 |
| RAG | retrieval-augmented generation: add retrieved documents to the prompt | §17.9 |
| Rate limiter | cap on calls per time period (Resilience4j, gateway filter) | §16.8 |
| Reactive Streams | spec: `Publisher`, `Subscriber`, `Subscription`, `Processor` with demand | §13.2 |
| Readiness probe | "send me traffic?" — may include essential dependencies | §15.3 |
| Refresh (context) | `refresh()` creates all non-lazy singletons and starts the context | §2.2 |
| `RefreshScope` | Spring Cloud scope re-creating beans after a configuration refresh | §16.3 |
| Relaxed binding | several spellings map to one property (`checkout-timeout`, `checkoutTimeout`, `SHOP_CHECKOUTTIMEOUT`) | §3.4 |
| Release train | set of versions released together (Spring Data 2026.0, Spring Cloud 2025.1) | §16.2 |
| Repository | Spring Data interface implemented at startup (`JpaRepository`, `CrudRepository`) | §8.8 |
| `REQUIRES_NEW` | propagation that suspends the caller's transaction and runs its own | §7.9 |
| Resilience4j | library for circuit breaker, retry, rate limiter, bulkhead, time limiter | §16.8 |
| Resource server | API that accepts access tokens (`oauth2ResourceServer`) | §11.3 |
| `ResponseEntity` | full control of status, headers and body in MVC | §5.4 |
| `ResponseEntityExceptionHandler` | advice base class mapping Spring MVC exceptions to `ProblemDetail` | §5.7 |
| `RestClient` | Framework 6.1 fluent blocking HTTP client | §6.2 |
| `RestTemplate` | legacy blocking HTTP client; deprecation announced | §6.9 |
| `RestTestClient` | Framework 7 fluent test client for MockMvc or a real server | §12.9 |
| `Retryable` | Framework 7 retry annotation (1 + `maxRetries` attempts) | §4.7 |
| `RetryableTopic` | spring-kafka non-blocking retries via retry topics | §14.15 |
| `RetryTemplate` | programmatic retry (Framework 7 `org.springframework.core.retry`) | §4.7 |
| Role | authority whose name starts with `ROLE_` | §10.6 |
| Role hierarchy | `ROLE_ADMIN > ROLE_USER`: higher roles include lower ones | §10.6 |
| Rollback-only | flag on a shared transaction that forces a rollback at commit | §7.9 |
| Rollback rules | unchecked → rollback, checked → commit by default | §7.10 |

## S

| Term | Meaning | See |
|---|---|---|
| Sampling (tracing) | share of traces recorded; Boot default `0.1` | §15.7 |
| Savepoint | point inside a transaction that can be rolled back to (`NESTED`) | §7.9 |
| `Scheduled` | method run on a timer (`fixedRate`, `fixedDelay`, `cron`) | §14.7 |
| Scheduler (Reactor) | thread pool abstraction: `parallel`, `boundedElastic`, `single` | §13.8 |
| Scope | how many bean instances exist and how long they live | §2.6 |
| Scoped proxy | stand-in that finds the request/session instance on every call | §2.6 |
| `securityMatcher` | which requests a whole `SecurityFilterChain` handles | §10.3 |
| `SecurityContextHolder` | `ThreadLocal` holder of the current `Authentication` | §10.2 |
| `SecurityFilterChain` | ordered list of security filters for matching requests | §10.3 |
| Self-invocation | `this.method()` inside a bean bypasses its proxy and all proxy features | §4.5 |
| `ServiceConnection` | Boot takes connection details from a Testcontainers container | §12.14 |
| Session fixation | attack reusing a pre-login session id; Spring changes the id at login | §10.10 |
| `SessionCreationPolicy` | `IF_REQUIRED`, `ALWAYS`, `NEVER`, `STATELESS` | §10.10 |
| ShedLock | library that lets a `@Scheduled` job run on one instance only | §14.7 |
| Singleton scope | one bean instance per container, shared by all threads (default) | §2.6 |
| Sinks | Reactor API to push values into a `Flux` / `Mono` from imperative code | §13.9 |
| Slice (test) | partial context for one layer (`@WebMvcTest`, `@DataJpaTest`...) | §12.7 |
| `SmartLifecycle` | beans started/stopped with the context, ordered by phase | §2.7 |
| Span | one timed operation in a trace, with trace id, span id and parent | §15.7 |
| SpEL | Spring Expression Language: `#{...}` in `@Value`, security and cache expressions | §3.5 |
| Spring AI | Spring project for LLM chat, embeddings, RAG, tools and MCP | §17.1 |
| Spring Boot | Spring + starters + managed versions + auto-configuration + embedded server + Actuator | §1.2 |
| Spring Cloud | projects for distributed systems: config, discovery, gateway, load balancing | §16.1 |
| Spring Data | family of repository modules sharing one programming model | §9.1 |
| Spring Data JDBC | aggregate-oriented mapping over plain JDBC, no persistence context | §9.2 |
| Spring Data REST | exports repositories as HAL REST endpoints | §9.3 |
| Spring Initializr | start.spring.io project generator | §1.5 |
| SSE | Server-Sent Events: one-way server push over `text/event-stream` | §13.12 |
| Starter | code-free dependency bringing one capability's libraries and auto-configuration | §1.7 |
| `StepVerifier` | reactor-test tool that asserts the signals of a publisher | §13.5 |
| Stereotype annotation | `@Component` and its specialisations `@Service`, `@Repository`, `@Controller` | §2.3 |
| Structured logging | JSON log lines (`ecs`, `logstash`, `gelf`), Boot 3.4+ | §15.8 |
| Structured output | converting an LLM answer into a Java type (`.entity(...)`) | §17.7 |
| `SpringBootApplication` | `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` | §1.8 |
| `SpringBootTest` | test that starts the whole application context | §12.6 |

**T**

| Term | Meaning | See |
|---|---|---|
| `TaskDecorator` | wraps submitted tasks to copy `ThreadLocal`s (MDC) to worker threads | §14.4 |
| `TestConfiguration` | configuration for tests that adds to (not replaces) the app's configuration | §12.6 |
| TestContext Framework | spring-test engine: `SpringExtension`, listeners, context cache | §12.5 |
| Testcontainers | library starting real services in Docker for tests | §12.14 |
| Thread-per-request | classic servlet model: one thread per request from start to end | §13.1 |
| Timeout (connect / read) | time to establish a connection / to wait for response data | §6.4 |
| Token relay | forwarding the user's access token to downstream services | §11.10 |
| Tool calling | the model requests a function; the application runs it (`@Tool`) | §17.8 |
| Trace / `traceparent` | a request's tree of spans / the W3C header that propagates it | §15.7 |
| `Transactional` | declarative transaction around a method (proxy-based) | §7.7 |
| `TransactionalEventListener` | listener bound to a transaction phase (`AFTER_COMMIT` default) | §7.15 |
| `TransactionTemplate` | programmatic transaction around a lambda | §7.14 |
| Transient entity | new object unknown to the persistence context | §8.4 |
| TTL | time to live of a cache entry; a provider setting, not an annotation attribute | §9.8 |

**U – W**

| Term | Meaning | See |
|---|---|---|
| `UnexpectedRollbackException` | commit of a transaction already marked rollback-only | §7.9 |
| `UserDetailsService` | loads a user by name for password authentication | §10.4 |
| Validation groups | subsets of constraints selected with `@Validated(Group.class)` | §5.6 |
| `Validated` | Spring annotation: validation groups; on a class, method validation via proxy | §5.6 |
| `Version` | optimistic-locking column (JPA and Spring Data) | §8.14 |
| Vector store | database of embeddings for similarity search (`SimpleVectorStore`, PgVector) | §17.9 |
| Virtual threads | cheap JVM-managed threads (Java 21); `spring.threads.virtual.enabled=true` | §13.15, §15.10 |
| Virtual time | `StepVerifier` clock that advances only when the test says so | §13.5 |
| Weaving | linking aspects with targets; Spring does it at runtime with proxies | §4.1 |
| `WebClient` | reactive, non-blocking HTTP client | §13.13 |
| WebFlux | Spring's reactive web framework on Reactor Netty | §13.10 |
| `WebMvcTest` | web-layer test slice | §12.7 |
| `WebTestClient` | fluent test client for WebFlux (and MockMvc) | §13.12 |

---

## Chapter TL;DR

- **Versions**: Boot 4.1.1 → Framework 7.0.9, Security 7.1.1, Hibernate 7.4, Jackson 3.1, JUnit 6, Java 17+ (25 used); Spring Cloud 2025.1, Spring AI 2.0 and springdoc 3 are versioned separately (§19.2).
- **Annotations** only work with their switch: `@EnableAsync`, `@EnableScheduling`, `@EnableCaching`, `@EnableMethodSecurity`, `@EnableResilientMethods`, `management.observations.annotations.enabled` — and only through the proxy (§19.3).
- **Starters** are modular in Boot 4: `-webmvc` (not `-web`), `-aspectj` (not `-aop`), `-security-oauth2-*`, `-restclient`, `-webclient`, `-flyway`, plus a `-test` starter per technology (§19.4).
- **Defaults worth remembering** (✔): Hikari pool 10 / 30 s wait, `@Async` pool 8 threads `task-`, one scheduler thread, 30 s shutdown phase, only `health` exposed, tracing sampling 0.1, OSIV on, graceful shutdown on (§19.5).
- **Transactions** (✔): `REQUIRED` joins, `REQUIRES_NEW` survives the caller's rollback, `NESTED` = savepoint, `MANDATORY`/`NEVER` throw; checked exceptions **commit**; a caught inner failure → `UnexpectedRollbackException` (§19.6).
- **Beans** (✔): singleton shared, prototype new per lookup and never destroyed; `@Qualifier` > `@Primary` > `@Fallback` > name; init `@PostConstruct` → `afterPropertiesSet` → `initMethod` (§19.7).
- **Security 7**: lambda DSL only; `hasRole("X")` = `ROLE_X`; 401 unknown vs 403 forbidden; CSRF on for cookies, `csrf.spa()` for SPAs; `http.cors()` before authentication (§19.8).
- **Tests**: unit → slice → a few `@SpringBootTest`s; `@MockitoBean` replaced `@MockBean`; explicit `@AutoConfigure...` for test clients in Boot 4; context cache max 32 (§19.9).

- **Choosing**: `RestClient`/HTTP interfaces for blocking calls, MVC + virtual threads for CRUD, WebFlux for streaming, JPA vs `JdbcClient` vs Data JDBC by model, Caffeine local vs Redis shared, Kafka log vs RabbitMQ queues, Resilience4j for circuit breakers (§19.12).
- **Upgrading**: 3.5 first, OpenRewrite, rename starters and properties, Jackson 3 imports, test starters, `@MockitoBean`, lambda-only security (§19.13).

This is the last chapter. Where to go from here:

- Spring Handbook front page — the chapter list, reading paths and how to run the code.
- Back to the start: Chapter 1 — The Spring Ecosystem and Your First App; revision questions: Chapter 18.
- Java Handbook — the language underneath Spring (records, generics, lambdas, concurrency and virtual threads).
- SQL Handbook — the queries, indexes and transaction isolation behind Chapters 7–9.
- Microservices patterns notes — gateway, discovery, circuit breaker, saga, CQRS and outbox, the designs Chapter 16 implements.